

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_b7f012f8-5c3\agent-a90073730cdbf577a.jsonl`  
- SHA-256: `49cf28d4051af17e6302a367af6eb7620d2877f60eb6f01f0dc00833da849ad9`  
- Source modified: `2026-06-18T01:31:06+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_b7f012f8-5c3`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:27:48.615Z)

You are an adversarial bug-hunter in a mature Python repo (badminton-highlight-indexer: FastAPI + SQLite + ffmpeg + GPU CV/AI rally detector). Read code READ-ONLY in this worktree (pinned to origin/master ? DO NOT modify anything):

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base

Hunt ONLY for REAL, REACHABLE defects a maintainer would agree is a bug and want fixed:

- crash on a reachable input (unguarded dict/index/attr access, None deref, KeyError/IndexError),
- wrong logic / off-by-one / inverted condition / wrong operator,
- incorrect error handling (catching too much/little, swallowing, wrong exception type),
- resource leak (unclosed file/db/subprocess), TOCTOU, mutable-default-arg,
- contract mismatch between a function and its callers/tests,
- wrong default / unit / sign that produces incorrect output.

HARD RULES:

- PRECISION over recall. Report a finding ONLY if you traced the actual code and a concrete trigger reaches it. No style, no "could be cleaner", no speculative "might", no perf-nits, no missing-test observations, no naming. If you are not sure it is a real bug, DROP it.
- For each finding give an EXACT file path + line, the bug_class, a CONCRETE trigger (the input or call path that hits it), why it is wrong, severity, and a minimal proposed fix.
- This repo has 848 passing tests and several hardening sweeps ? easy bugs are already gone. Expect FEW or ZERO real bugs in your area. Returning an empty candidates list is a VALID, respected answer. Do NOT manufacture findings to look productive.
- EXCLUDE these files entirely (just fixed / fix already in flight): backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py.
- Verify your trigger by reading the surrounding code and any related test, not from memory.

YOUR AREA: backend/eval/ golden/feature substrate ?

golden_fixtures.py, fusion_golden.py, serve_contrast.py, distill_local.py, tracknet_runner.py, rally_gate.py, and backend/eval/audio/*. Focus on the windowing/trim/cap logic (R3/R4/R5 knobs: _trim_inactive_tail, _blob_fallback_chop, hard_cap), JSON schema read/write, and off-by-one in frame/time conversions.

2. assistant (2026-06-18T01:27:50.971Z)

I'll hunt for bugs in the specified area. Let me start by reading the target files.

3. user (2026-06-18T01:27:52.683Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.  
2  
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per  
shuttle  
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
```

```

5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42         if spec.get("video_path"):
43             return str(spec["video_path"])
44         labels = str(spec["labels"])
45         if labels.endswith(".rallies.csv"):
46             return labels[: -len(".rallies.csv")] + ".MP4"
47         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                     frame_skip: int = 3, iou: float = 0.5,
52                     log_every_sec: float = 15.0,
53                     telemetry: Optional[Any] = None) -> Dict[str, Any]:
54         """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56         Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle{}, person{}, label}], gts}`.
57         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;

```

```

58         when None a real `PersonDetector` is built (loads torchvision on first use).
59         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60         from the detection progress callback so a hard kill leaves a forensic runlog.
61         """
62         fps, fw = float(spec["fps"]), float(spec["frame_width"])
63         traj = str(spec["trajectory"])
64         points = TrackNetRunner.parse_trajectory_csv(traj)
65         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66         gts = load_golden_gts(str(spec["labels"]))
67         labels = label_candidates(intervals, gts, iou)
68         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69         # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70         axis, net_px, _span = rally_gate.estimate_play_axis(points)
71         video_path = _derive_video_path(spec)
72
73         if detector is None:
74             from backend.pipeline.detectors.person_detector import PersonDetector
75             detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79         n = len(intervals)
80         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81         print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82         t0 = time.monotonic()
83         state = {"last": t0}
84
85         def _progress(done: int, total: int) -> None:
86             if telemetry is not None:
87                 telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                         f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96                 state["last"] = now
97
98         per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                         progress_cb=_progress)
100
101         candidates: List[Dict[str, Any]] = []
102         for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103             det = per_window[i]
104             det_fw = det.get("frame_width") or fw
105             det_fh = det.get("frame_height") or 0.0
106             # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107             # normalisation fusion_features uses (x/width, y/height).
108             if axis == "x":
109                 net_norm = net_px / det_fw if det_fw > 0 else 0.5
110             else:
111                 net_norm = net_px / det_fh if det_fh > 0 else 0.5
112             sf, ef = win_frames[i]
113             shuttle_in = [p for p in points if sf <= p.frame <= ef]
114             person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                                 court_polygon=None, net=(axis, net_norm))
116             candidates.append({

```

```

117         "start": float(s), "end": float(e),
118         "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119         "person": {k: float(v) for k, v in person.items()},
120         "label": int(labels[i]),
121     })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
136             # row/frame).
137             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
138                                                  if os.path.isfile(s.get("trajectory", ""))
139                                                  else 0))
140         os.makedirs(out_dir, exist_ok=True)
141         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
142         # keep a tiny
143         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
144         # must never
145         # break extraction, so guard the build and fall back to no telemetry on any error.
146         telem: Optional[RunTelemetry]
147         try:
148             telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),
149                                  label="fusion_golden")
150         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
151             # observes
152             telem = None
153         written: List[str] = []
154         for vi, spec in enumerate(specs, 1):
155             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
156             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
157             # prior
158             # run left off instead of re-detecting). --fresh forces a redo.
159             if not fresh and os.path.isfile(out):
160                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
161                 extracted "
162                 f"-> {out} (use --fresh to redo)", flush=True)
163                 written.append(out)
164                 continue
165             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
166                  flush=True)
167             t0 = time.monotonic()
168             if telem is not None:
169                 telem.event("video_start", spec["name"])
170             try:
171                 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
172             except Exception as e: # noqa: BLE001 - record machine state at death, then re-
173                 raise
174             if telem is not None:
175                 telem.event("error", spec["name"], err=str(e))
176             raise
177             n_pos = sum(c["label"] for c in data["candidates"])
178             with open(out, "w", encoding="utf-8") as f:
179                 json.dump(data, f, indent=2)
180             if telem is not None:

```

```

171         telem.event("video_done", spec["name"],
172                     windows=len(data["candidates"]), positives=n_pos)
173         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
174             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175             f"-> {out}", flush=True)
176         written.append(out)
177     return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

4. user (2026-06-18T01:27:52.910Z)

```

1     """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3     `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the **single shared
4     replay code path** used by both the freeze tool and the characterization gate, and it
5     deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6     variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7     across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8     only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9     merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10    LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11    `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12    path simply never calls into it.)
13
14    The replay reuses the already-shipped eval stack VERBATIM::
15
16        parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17        ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18        ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20    Tier-0 / privacy invariants (D3, $4c, $6 of the doc), enforced here by construction:
21
22    * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23    provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24    * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"`.

```

```

25      * ``expected.json`` carries only the 5 shuttle feature columns ? there is, by
design, no
26      person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27      * The windowing params ``(vt, mg, mwd)`` are pinned inside each fixture's provenance
and
28      passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29      named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30      silently churn fixtures.
31      * ``max_win`` (W1 bounded-windowing) is RECORDED in provenance/manifest but NOT
applied by
32      the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33      takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34      has no max-window arg, so these fixtures characterize the W1-OFF path by design.
The pinned
35      ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36      tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38      $6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39      suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40      frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41      that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42      field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44      Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45      parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46      NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47      the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48      never calls into it.
49      """
50      from __future__ import annotations
51
52      import argparse
53      import json
54      import math
55      import os
56      import re
57      import shutil
58      import sys
59      import tempfile
60      from dataclasses import dataclass
61      from pathlib import Path
62      from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64      # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
65      from backend.eval import distill_local
66      from backend.eval.experiment import CONTROL, VideoCandidates, predict
67      from backend.eval.gt_loader import load_gt_intervals
68      from backend.eval.metrics import score
69      from backend.eval.regression import gt_hash
70      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
71
72      Interval = Tuple[float, float]

```

```

73
74 #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
75 #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
76 CURRENT_SCHEMA = 1
77
78 #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
79 #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
80 FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
81
82 #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
83 #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
84 #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
85 #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
86 #: re-states these in its provenance; the replay reads them from there.
87 PINNED_VT = 0.02
88 PINNED_MERGE_GAP = 2.0
89 PINNED_MIN_WINDOW = 2.0
90 #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
91 #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
92 #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
93 #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
94 #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
95 PINNED_MAX_WINDOW = 0.0
96
97 #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
98 #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
99 SHUTTLE_FEATURES: Tuple[str, ...] = (
100     "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
101 )
102 assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
103     "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
104 )
105
106 #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
107 #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
108 FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
109
110 _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
111 _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
112
113 #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
114 #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
115 _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
116
117 #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
118 #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
119 #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a

```

```

FORBIDDEN_KEY / MD5
120     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
121     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
122
123
124     # =====
125     # Synthetic trajectory generation (deterministic ? NO random)
126     # =====
127
128
129     @dataclass(frozen=True)
130     class RallySpec:
131         """One in-flight rally segment: the shuttle oscillates across the net midline.
132
133         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
134         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
135         net midline along the play axis. Deterministic sine motion ? fixed
136         velocity/crossings.
137         """
138         start_frame: int
139         n_frames: int
140         period: float = 30.0
141         amplitude: float = 400.0
142
143
144     @dataclass(frozen=True)
145     class GapSpec:
146         """Dead-time between rallies ? what separates one window from the next.
147
148         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
149         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
150         end an active run so the next rally forms a distinct window.
151         """
152
153         start_frame: int
154         n_frames: int
155         kind: str = "still" # "still" | "occluded"
156
157
158     @dataclass(frozen=True)
159     class FixtureSpec:
160         """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
161
162         ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
163         "labels.csv". For a synthetic fixture they line up with the rallies by construction,
164         which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
165         field-quality one (§6).
166         """
167
168         slug: str
169         description: str
170         fps: float
171         frame_width: float
172         frame_height: float
173         segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
174         gts: Tuple[Interval, ...]
175         net_midline: float = 640.0
176
177
178     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
179                       ) -> List[Tuple[int, int, float, float]]:
180         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""

```



```

181     rows: List[Tuple[int, int, float, float]] = []
182     y_mid = frame_height / 2.0
183     for i in range(spec.n_frames):
184         x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
185         # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
186         y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
187         rows.append((spec.start_frame + i, 1, x, y))
188     return rows
189
190
191 def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
192                ) -> List[Tuple[int, int, float, float]]:
193     """Dead-time rows: resting-visible (still) or absent (occluded)."""
194     rows: List[Tuple[int, int, float, float]] = []
195     y_mid = frame_height / 2.0
196     for i in range(spec.n_frames):
197         if spec.kind == "occluded":
198             rows.append((spec.start_frame + i, 0, 0.0, 0.0))
199         elif spec.kind == "still":
200             rows.append((spec.start_frame + i, 1, net_midline, y_mid))
201         else: # pragma: no cover - guarded by builder
202             raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
203     return rows
204
205
206 def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
207     """Build the deterministic ``Frame, Visibility, X, Y`` rows for a synthetic
scenario."""
208     rows: List[Tuple[int, int, float, float]] = []
209     for seg in spec.segments:
210         if isinstance(seg, RallySpec):
211             rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
212         elif isinstance(seg, GapSpec):
213             rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
214         else: # pragma: no cover - guarded by dataclass shape
215             raise TypeError(f"unknown segment type {type(seg).__name__}")
216     rows.sort(key=lambda r: r[0])
217     return rows
218
219
220 def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
221     """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
floats."""
222     lines = ["Frame,Visibility,X,Y"]
223     for frame, vis, x, y in rows:
224         lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
225     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
226
227
228 def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
229     """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
230     lines = ["start,end"]
231     for s, e in gts:
232         lines.append(f"{_fmt(s)},{_fmt(e)}")
233     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
234
235
236 # =====
237 # Built-in synthetic scenarios (the 3 committed slugs)
238 # =====
239
240

```

```

241     def _fr(seconds: float, fps: float) -> int:
242         return int(round(seconds * fps))
243
244
245     def builtin_specs() -> List[FixtureSpec]:
246         """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
247         windowing behaviour (one window / dead-time split / occlusion split)."""
248         fps = 30.0
249         fw, fh = 1280.0, 720.0
250         net = 640.0
251
252         # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
253         single = FixtureSpec(
254             slug="clean_single_rally",
255             description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
256             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
257             segments=(RallySpec(start_frame=0, n_frames=120),),
258             # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
259             gts=((0.0, 4.0),),
260         )
261
262         # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
263         # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
264         r0 = RallySpec(start_frame=0, n_frames=120)
265         gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
266         r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
267         multi = FixtureSpec(
268             slug="multi_rally_with_deadtime",
269             description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
270             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
271             segments=(r0, gap_still, r1),
272             gts=((0.0, 4.0), (7.0, 11.0)),
273         )
274
275         # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
276         # visibility break ? proves occlusion (not just low velocity) separates windows.
277         o0 = RallySpec(start_frame=0, n_frames=120)
278         gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
279         o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
280         occ = FixtureSpec(
281             slug="occlusion_break",
282             description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
283             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
284             segments=(o0, gap_occ, o1),
285             gts=((0.0, 4.0), (7.0, 11.0)),
286         )
287
288         return [single, multi, occ]
289
290
291     def _spec_by_slug() -> Dict[str, FixtureSpec]:
292         return {s.slug: s for s in builtin_specs()}
293
294
295     # =====
296     # The shared replay (CONTROL / pure-stdlib path)
297     # =====
298

```

```

299
300     def _fmt(x: float) -> str:
301         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
302         ``0.0``."""
303         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
304         if r == 0.0:
305             r = 0.0
306         return f"{r:.6f}"
307
308     def _round(x: float) -> float:
309         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
310         return 0.0 if r == 0.0 else r
311
312
313     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
314         """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
315         when present
316         (the pinned params travel with the fixture), else the module-level pinned defaults.
317         NEVER
318         ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
319         read here ?
320         ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
321         path; the
322         pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
323         PINNED_MAX_WINDOW)."""
324         if prov and "windowing" in prov:
325             w = prov["windowing"]
326             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
327         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
328
329
330     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
331         """Replay one fixture through the shipped CONTROL path; return its recomputed
332         snapshot.
333         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
334         committed
335         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
336
337         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
338         shuttle
339         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
340
341         Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
342         control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
343         then
344         merges overlaps; NO numpy is touched.
345         """
346         fdir = _resolve_dir(slug_or_dir)
347         slug = fdir.name
348         traj_csv = fdir / "trajectory.csv"
349         labels_csv = fdir / "labels.csv"
350         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
351         else None
352
353         fps, fw = _fps_fw(prov, slug)
354         iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
355         proposal = _pinned_proposal(prov)
356
357         intervals, feature_rows = distill_local.build_candidates(
358             str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
359
360         # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
361         call

```

```

352         # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353         # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354         features: Dict[str, List[float]] = {
355             name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356         }
357         gts = load_gt_intervals(str(labels_csv), strict=True)
358         vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360         preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361         sc = score(preds, gts, iou)
362
363         candidates: List[Dict[str, Any]] = []
364         for j, (s, e) in enumerate(intervals):
365             shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
366             # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
367             shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
368             candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
369
370         return {
371             "slug": slug,
372             "schema_version": CURRENT_SCHEMA,
373             "iou": _round(iou),
374             "candidates": candidates,
375             "control_segments": [[_round(s), _round(e)] for s, e in preds],
376             "score": {
377                 "iou_threshold": _round(sc.iou_threshold),
378                 "num_pred": int(sc.num_pred),
379                 "num_gt": int(sc.num_gt),
380                 "true_positives": int(sc.true_positives),
381                 "false_positives": int(sc.false_positives),
382                 "false_negatives": int(sc.false_negatives),
383                 "precision": _round(sc.precision),
384                 "recall": _round(sc.recall),
385                 "f1": _round(sc.f1),
386                 "mean_iou": _round(sc.mean_iou),
387                 "mean_start_error": _round(sc.mean_start_error),
388                 "mean_end_error": _round(sc.mean_end_error),
389             },
390             "gt_hash": gt_hash(gts),
391         }
392
393
394         # =====
395         # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
396         # =====
397
398
399         def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
400             ) -> Dict[str, Any]:
401             """The committed provenance block ? pins the windowing + privacy/licensing tags
($4c, §5)."""
402             desc = ""
403             if isinstance(spec_or_dir, FixtureSpec):
404                 desc = spec_or_dir.description
405             return {
406                 "slug": slug,
407                 "schema_version": CURRENT_SCHEMA,
408                 "kind": "synthetic",
409                 "license": "synthetic",

```

```

410         "minors_free": True,
411         "description": desc,
412         "fps": _round(fps),
413         "frame_width": _round(fw),
414         "iou": _round(iou),
415         "windowing": {
416             "vt": PINNED_VT,
417             "mg": PINNED_MERGE_GAP,
418             "mwd": PINNED_MIN_WINDOW,
419             "max_win": PINNED_MAX_WINDOW,
420         },
421         "paths": {
422             "trajectory": "trajectory.csv",
423             "labels": "labels.csv",
424             "expected": "expected.json",
425         },
426         "note": (
427             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
428             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
429         ),
430     }
431
432
433     def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
434                       iou: float = 0.5) -> Dict[str, Any]:
435         """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
436         ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
437
438         Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
439         + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
440         """
441         fdir = fixtures_dir / slug
442         traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
443         if not traj_csv.is_file() or not labels_csv.is_file():
444             raise FileNotFoundError(
445                 f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
446
447         # Provenance first (it pins the windowing the replay reads), then replay ? expected.
448         spec = _spec_by_slug().get(slug)
449         fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
450         prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
451         _write_json(fdir / "provenance.json", prov)
452
453         expected = replay_fixture(fdir)
454         _write_json(fdir / "expected.json", expected)
455         return expected
456
457
458     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
459         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
460         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
461         specs = builtin_specs()
462         fixtures_dir.mkdir(parents=True, exist_ok=True)
463         slugs: List[str] = []
464         for spec in specs:
465             fdir = fixtures_dir / spec.slug
466             fdir.mkdir(parents=True, exist_ok=True)
467             rows = make_synthetic_trajectory(spec)

```

```

468         write_trajectory_csv(rows, fdir / "trajectory.csv")
469         write_labels_csv(spec.gts, fdir / "labels.csv")
470         freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
471         slugs.append(spec.slug)
472     _write_manifest(specs, fixtures_dir)
473     return slugs
474
475
476     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
477                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
478         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
479         labels.csv ?
480         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
481         regression; the
482         mitigation is reviewing the visible diff + recording a reason (the CLI requires
483         one)."""
484         if all_slugs:
485             targets = [d.name for d in sorted(fixtures_dir.iterdir())
486                       if d.is_dir() and (d / "trajectory.csv").is_file()]
487         elif slug:
488             targets = [slug]
489         else:
490             raise ValueError("refresh_snapshot needs --slug S or --all")
491         for s in targets:
492             freeze_fixture(s, fixtures_dir=fixtures_dir)
493         return targets
494
495     # =====
496     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
497     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
498     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
499     # =====
500
501     class RefusalError(ValueError):
502         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
503
504         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
505         it) that
506         names *why* the unsafe public-fixture path was refused. The refusal is the code-
507         enforced
508         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
509         minors-excluded
510         / license flags are human-asserted, but the gate makes the unsafe path impossible to
511         reach
512         *silently* ? a fixture cannot be written unless all three are affirmatively set.
513         """
514
515     def _new_surrogate_slug() -> str:
516         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
517
518         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
519         random,
520         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
521         over a
522         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
523         off-git.
524         """
525         return f"fx_{os.urandom(8).hex()}"
526
527     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
528     path token

```

```

522     #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
regardless), so it is
523     #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
would let
524     #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
(red-team #4).
525     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
526         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
527         "provenance", "provenance.json",
528     )
529
530
531     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
532         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.
533
534         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
535         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
536         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
537         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
538         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
539         """
540         low = text.lower()
541         for key in FORBIDDEN_KEYS:
542             if key in low:
543                 raise RefusalError(
544                     f"{where}: forbidden key substring {key!r} found in written output ? "
545                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
546                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
547         m = _MD5_RE.search(text)
548         if m:
549             raise RefusalError(
550                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
551                 f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
552         for sp in source_paths:
553             if not sp:
554                 continue
555             p = Path(sp)
556             for token in (sp, p.name, p.stem):
557                 t = (token or "").strip().lower()
558                 # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
559                 # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
560                 # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
561                 # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
562                 # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
563                 if not t or t in _SCHEMA_FILENAME_LITERALS:
564                     continue
565                 if t in low:
566                     raise RefusalError(
567                         f"{where}: source-path token {token!r} found in written output ? the
source "
568                         f"filename/path must never be persisted (§4c metadata strip).")
569
570

```

```

571     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
572                                   ) -> List[Tuple[int, int, float, float]]:
573         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
574         ``(Frame,Visibility,X,Y)`` rows.
575         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
576         invariant under a
577         pure time shift); only the absolute frame index moves toward 0, severing the match
578         timeline.
579         """
580         rows: List[Tuple[int, int, float, float]] = []
581         for p in points:
582             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
583             float(p.y)))
584         rows.sort(key=lambda r: r[0])
585         return rows
586
587     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
588         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
589         invariant).
590         ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
591         Each label
592         endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
593         of two exact
594         6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
595         ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
596         reset score
597         delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
598         ``freeze_real_fixture``
599         for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
600         second
601         offset (the grid, not a real metric divergence)."""
602         return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]
603
604     def freeze_real_fixture(
605         label: str,
606         trajectory_csv_path: Any,
607         labels_csv_path: Any,
608         *,
609         license: str,
610         minors_free: bool,
611         owned: bool,
612         fps: float,
613         frame_width: float,
614         fixtures_dir: Path = FIXTURES_DIR,
615         slug: Optional[str] = None,
616         time_origin_reset: bool = True,
617         source_note: str = "",
618     ) -> Dict[str, Any]:
619         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
620         de-attributed
621         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
622         row P1.)
623
624         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
625         is emitted
626         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
627         blank string,
628         AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text
629         identifier like a
630         player/event name is refused, red-team #1). The committed bytes carry NO filename /
631         video_id /

```



```

620 match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
621 shuttle-only trajectory (time-origin reset by default), the reset labels, and a
622 ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
623 ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.
624
625 ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
626 back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
627 contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
628 committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
629 retained for CLI compatibility / future logging.
630
631 NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
632 only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
633 attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
634 owner-asserted Fork-A / minors / biometric-exclusion flags.
635
636 The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
637 after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
638 (red-team #3).
639
640 Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
641 """
642 # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
643 reasons: List[str] = []
644 if minors_free is not True:
645     reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
646                     "of a minor is prohibited ? $3, guardrail 'exclude minors')")
647 if owned is not True:
648     reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
649                     "broadcast/scraped = Fork B, do NOT commit ? $3 D1)")
650 if not (isinstance(license, str) and license.strip()):
651     reasons.append("license must be a non-blank string (per-record provenance tag ?
$4c "
652                     "'provenance hard-fail gate')")
653 # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
654 # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
655 if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
656     reasons.append(
657         f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}
(lowercase "
658         f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier
(player/event "
659         f"name) is an attribution leak; got {label!r} ($4c metadata strip, red-team
#1)")
660 if reasons:
661     raise RefusalError(
662         "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution

```

```

does NOT "
663         "cure provenance/licensing; this gate sits behind counsel sign-off,
$6/P2):\n - "
664         + "\n - ".join(reasons))
665
666     traj_path = Path(str(trajjectory_csv_path))
667     labels_path = Path(str(labels_csv_path))
668
669     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
670     out_slug = slug if slug is not None else _new_surrogate_slug()
671
672     # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
-----
673     points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
674     if not points:
675         raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
676     gts = load_gt_intervals(str(labels_path), strict=True)
677
678     # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, $4c) --
679     # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
680     # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
681     # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
682     # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
683     # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
684     # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
685     #
686     # HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while
labels shift
687     # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
688     # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
689     # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
690     # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).
691     if time_origin_reset:
692         t0_frame = min(int(p.frame) for p in points)
693         t0_sec = _round(t0_frame / float(fps))
694     else:
695         t0_frame, t0_sec = 0, 0.0
696     rows = _reset_trajectory_rows(points, t0_frame)
697     reset_gts = _reset_labels(gts, t0_sec)
698
699     # --- (d2) negative-label guard BEFORE any write (red-team #3a)
-----
700     # A reset label that starts < 0 means the label timeline predates the earliest
trajectory
701     # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
702     # start). Refuse loudly up-front so the partial-write path is never even reached.
703     neg = [(s, e) for (s, e) in reset_gts if s < 0]
704     if neg:
705         raise RefusalError(
706             f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "

```

```

707         f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
708         f"earliest label must not precede the earliest trajectory frame (§4c time-
origin reset).")
709
710     iou = 0.5
711     # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
712     # it never reaches the committed bytes (an identifying note is an attribution back-
channel
713     # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
714     prov = {
715         "slug": out_slug,
716         "schema_version": CURRENT_SCHEMA,
717         "kind": "cleared_real",
718         "label": str(label),
719         "license": license,
720         "minors_free": True,
721         "owned": True,
722         "fps": _round(fps),
723         "frame_width": _round(frame_width),
724         "iou": _round(iou),
725         "time_origin_reset": bool(time_origin_reset),
726         "windowing": {
727             "vt": PINNED_VT,
728             "mg": PINNED_MERGE_GAP,
729             "mwd": PINNED_MIN_WINDOW,
730             "max_win": PINNED_MAX_WINDOW,
731         },
732         "paths": {
733             "trajectory": "trajectory.csv",
734             "labels": "labels.csv",
735             "expected": "expected.json",
736         },
737         "gt_hash": gt_hash(reset_gts),
738         "note": (
739             "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
740             "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
741             "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
742             "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2).",
743         ),
744     }
745
746     # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
-----
747     # Everything is written under a sibling temp dir; the final slug dir is created by
an atomic
748     # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
leaves NO
749     # partial fixture dir behind. source_note (if any) is scanned defensively but never
written.
750     fdir = fixtures_dir / out_slug
751     fixtures_dir.mkdir(parents=True, exist_ok=True)
752     if fdir.exists():
753         raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
(choose a new slug)")
754     tmp_parent = Path(tempfile.mkdtemp(prefix=f"{out_slug}.tmp.",
dir=str(fixtures_dir)))
755     tmp_dir = tmp_parent / out_slug
756     tmp_dir.mkdir(parents=True, exist_ok=True)

```

```

757     try:
758         write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
759         write_labels_csv(reset_gts, tmp_dir / "labels.csv")
760         _write_json(tmp_dir / "provenance.json", prov)
761
762         expected = replay_fixture(tmp_dir)    # REUSE M1 ? correct-by-construction
snapshot
763         _write_json(tmp_dir / "expected.json", expected)
764
765         # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
source_note) ---
766         source_paths = [str(traj_path), str(labels_path)]
767         _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
768                         where=f"{out_slug}/expected.json", source_paths=source_paths)
769         _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
770                         where=f"{out_slug}/provenance.json", source_paths=source_paths)
771         # source_note is never persisted, but a leak in it is still a contract violation
? scan it.
772         if source_note:
773             _scan_forbidden(str(source_note),
774                             where=f"{out_slug}/source_note(not-persisted)",
source_paths=source_paths)
775
776         # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
-----
777         os.replace(str(tmp_dir), str(fdir))
778     finally:
779         # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
partial dir).
780         shutil.rmtree(str(tmp_parent), ignore_errors=True)
781
782         # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
783         _append_manifest_entry(prov, fixtures_dir)
784         return expected
785
786
787     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
788         """A manifest row for a single fixture, derived from its provenance block."""
789         slug = prov["slug"]
790         return {
791             "slug": slug,
792             "fps": _round(float(prov["fps"])),
793             "frame_width": _round(float(prov["frame_width"])),
794             "windowing": dict(prov.get("windowing", {})),
795             "kind": prov.get("kind", "synthetic"),
796             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
797             "paths": {
798                 "dir": slug,
799                 "trajectory": f"{slug}/trajectory.csv",
800                 "labels": f"{slug}/labels.csv",
801                 "expected": f"{slug}/expected.json",
802                 "provenance": f"{slug}/provenance.json",
803             },
804         }
805
806
807     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
808         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
809         entries = load_manifest(fixtures_dir)
810         entries = [e for e in entries if e.get("slug") != prov["slug"]]
811         entries.append(_manifest_entry_for(prov))
812         entries.sort(key=lambda e: str(e.get("slug")))
813         _write_json(fixtures_dir / "manifest.json", entries)

```

```

814
815
816 # =====
817 # Manifest + loaders
818 # =====
819
820
821 def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
822     entries = [{
823         "slug": spec.slug,
824         "fps": _round(spec.fps),
825         "frame_width": _round(spec.frame_width),
826         "windowing": {
827             "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
828             "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
829         },
830         "kind": "synthetic",
831         "schema_version": CURRENT_SCHEMA,
832         "paths": {
833             "dir": spec.slug,
834             "trajectory": f"{spec.slug}/trajectory.csv",
835             "labels": f"{spec.slug}/labels.csv",
836             "expected": f"{spec.slug}/expected.json",
837             "provenance": f"{spec.slug}/provenance.json",
838         },
839     ] for spec in specs]
840     _write_json(fixtures_dir / "manifest.json", entries)
841
842
843 def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
844     """Load the fixtures manifest (returns [] if absent)."""
845     path = fixtures_dir / "manifest.json"
846     if not path.is_file():
847         return []
848     data = _read_json(path)
849     if not isinstance(data, list): # pragma: no cover - corrupt manifest
850         raise ValueError(f"{path}: manifest must be a JSON array")
851     return data
852
853
854 def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
855     """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
856     whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
857     ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
858     bump in one is never masked by an in-range value in the other (red-team #7)."""
859     fdir = fixtures_dir / slug
860     prov = _read_json(fdir / "provenance.json")
861     expected = _read_json(fdir / "expected.json")
862     for which, blob in (("expected", expected), ("provenance", prov)):
863         sv = int(blob.get("schema_version", 0))
864         if sv > CURRENT_SCHEMA:
865             raise ValueError(
866                 f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
867                 f"upgrade the harness")
868     return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
869
870
871 # =====
872 # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
873 # =====
874

```

```

875
876     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
877         """Return a list of human-readable mismatch messages (empty == match).
878
879         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
880         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
881         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
882         safe (the repo runs with core.autocrlf=true).
883
884         Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
885         per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
886         a fixed-field loop alone would ignore an added field (red-team #5)."""
887         out: List[str] = []
888
889         def exact(path: str, a: Any, b: Any) -> None:
890             if a != b:
891                 out.append(f"{path}: {a!r} != {b!r} (exact)")
892
893         def approx(path: str, a: Any, b: Any) -> None:
894             try:
895                 if abs(float(a) - float(b)) > _FLOAT_TOL:
896                     out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
897             except (TypeError, ValueError):
898                 out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
899
900         # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
901         if set(recomputed) != set(committed):
902             out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
903
904         exact("slug", recomputed.get("slug"), committed.get("slug"))
905         exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
906         exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
907         approx("iou", recomputed.get("iou"), committed.get("iou"))
908
909         rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
910         if len(rc) != len(cc):
911             out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
912         else:
913             for i, (r, c) in enumerate(zip(rc, cc)):
914                 approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
915                 approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
916                 rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
917                 if set(rs) != set(cs):
918                     out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
919                 else:
920                     for k in rs:
921                         if k == "net_crossings":
922                             exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
923                         else:
924                             approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
925
926         rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
927         if len(rseg) != len(cseg):
928             out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")

```

```

929     else:
930         for i, (r, c) in enumerate(zip(rseg, cseg)):
931             approx(f"control_segments[{i}][0]", r[0], c[0])
932             approx(f"control_segments[{i}][1]", r[1], c[1])
933
934     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
935     # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
936     if set(rsc) != set(csc):
937         out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
938     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
939     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
940                     "mean_start_error", "mean_end_error")
941     for k in int_fields:
942         exact(f"score.{k}", rsc.get(k), csc.get(k))
943     for k in float_fields:
944         approx(f"score.{k}", rsc.get(k), csc.get(k))
945
946     return out
947
948
949 # =====
950 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
951 # =====
952
953
954 def _write_json(path: Path, obj: Any) -> None:
955     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
956     path.write_text(text + "\n", encoding="utf-8", newline="\n")
957
958
959 def _read_json(path: Path) -> Any:
960     with open(path, encoding="utf-8") as f:
961         return json.load(f)
962
963
964 def _resolve_dir(slug_or_dir: Any) -> Path:
965     if isinstance(slug_or_dir, Path):
966         return slug_or_dir
967     p = Path(str(slug_or_dir))
968     if p.is_dir():
969         return p
970     return FIXTURES_DIR / str(slug_or_dir)
971
972
973 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
974     if prov and "fps" in prov and "frame_width" in prov:
975         return float(prov["fps"]), float(prov["frame_width"])
976     spec = _spec_by_slug().get(slug)
977     if spec is not None:
978         return spec.fps, spec.frame_width
979     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
980
981
982 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
983     prov_path = fdir / "provenance.json"
984     prov = _read_json(prov_path) if prov_path.is_file() else None
985     return _fps_fw(prov, slug)
986
987
988 # =====
989 # CLI
990 # =====

```

```

991
992
993     def _cmd_check(fixtures_dir: Path) -> int:
994         manifest = load_manifest(fixtures_dir)
995         if not manifest:
996             print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
997             return 1
998         failures = 0
999         for entry in manifest:
1000             slug = entry["slug"]
1001             committed = _read_json(fixtures_dir / slug / "expected.json")
1002             recomputed = replay_fixture(fixtures_dir / slug)
1003             mismatches = compare_expected(recomputed, committed)
1004             if mismatches:
1005                 failures += 1
1006                 print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
1007                     f"(characterization = behaviour-locking, NOT correctness ? $6):")
1008                 for m in mismatches:
1009                     print(f"    - {m}")
1010             else:
1011                 sc = recomputed["score"]
1012                 print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
1013                     f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
1014             print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
1015                 f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
1016         return 0 if failures == 0 else 1
1017
1018
1019     def main(argv: Optional[Sequence[str]] = None) -> int:
1020         ap = argparse.ArgumentParser(
1021             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
1022         g = ap.add_mutually_exclusive_group(required=True)
1023         g.add_argument("--freeze-synthetic", action="store_true",
1024             help="generate synthetic trajectory+labels for all built-in
scenarios, then "
1025                 "freeze expected.json+provenance.json + write manifest.json")
1026         g.add_argument("--refresh-snapshot", action="store_true",
1027             help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
1028         g.add_argument("--check", action="store_true",
1029             help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
1030         g.add_argument("--freeze-real", action="store_true",
1031             help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
1032                 "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
1033         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
1034                 "--freeze-real (default: a random fx_<hex> id)")
1035         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
1036         ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
1037         # --freeze-real (P1 / M2 refusal gate) options:
1038         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
1039         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
1040         ap.add_argument("--license", dest="license_", default=None,
1041             help="--freeze-real) the source license tag (required, non-blank)")
1042         ap.add_argument("--owned", action="store_true",

```



```

1043             help="--freeze-real) assert owner-recorded Fork-A source
(required)")
1044         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1045             help="--freeze-real) assert the source contains NO minors
(required)")
1046         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
1047         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
1048         ap.add_argument("--label", default="cleared_real",
1049             help="--freeze-real) non-identifying scenario label ? STRICT slug
only "
1050             "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
1051         ap.add_argument("--source-note", dest="source_note", default="",
1052             help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
1053             "attribution back-channel, so it never reaches the committed
provenance "
1054             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
1055         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1056             help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
1057         ap.set_defaults(time_origin_reset=True)
1058         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1059             help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
1060         args = ap.parse_args(list(argv) if argv is not None else None)
1061
1062         fixtures_dir: Path = args.fixtures_dir
1063
1064         if args.freeze_synthetic:
1065             slugs = freeze_synthetic(fixtures_dir)
1066             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
1067             print(f" -> {fixtures_dir}")
1068             return 0
1069
1070         if args.refresh_snapshot:
1071             if not args.slug and not args.all:
1072                 ap.error("--refresh-snapshot requires --slug S or --all")
1073             # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
1074             # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
1075             if not (args.reason and args.reason.strip()):
1076                 ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
1077                     "regression; record the deliberate reason and review the visible
diff ($6).")
1078             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
1079             print(f"[golden-fixtures] refreshed snapshot for: {'', '.join(targets)}")
1080             f" (reason: {args.reason})")
1081             return 0
1082
1083         if args.freeze_real:
1084             missing = [n for n, v in (("--trajectory", args.trajectory),
1085                                     ("--labels", args.labels),
1086                                     ("--fps", args.fps),
1087                                     ("--frame-width", args.frame_width)) if v is None]
1088             if missing:
1089                 ap.error("--freeze-real requires " + ", ".join(missing))
1090             try:
1091                 expected = freeze_real_fixture(

```

```

1092         args.label, args.trajectory, args.labels,
1093         license=args.license_ if args.license_ is not None else "",
1094         minors_free=args.minors_free, owned=args.owned,
1095         fps=args.fps, frame_width=args.frame_width,
1096         fixtures_dir=fixtures_dir, slug=args.slug,
1097         time_origin_reset=args.time_origin_reset, source_note=args.source_note)
1098     except RefusalError as e:
1099         # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
1100         zero.
1101         print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
1102         file=sys.stderr)
1103         return 2
1104     print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
1105     "
1106         f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
1107     print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
1108     to the "
1109         "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
1110     (P2).")
1111     return 0
1112     # --check
1113     return _cmd_check(fixtures_dir)
1114
1115 if __name__ == "__main__":
1116     raise SystemExit(main())
1117

```

5. assistant (2026-06-18T01:27:55.419Z)

Let me read the remaining target files.

6. user (2026-06-18T01:27:56.371Z)

```

1      """The eval?serve worked example: score a cue at BOTH windowings and run the promotion
2      guard.
3      This is the reproducible proof of the foundational fix. For the Gate-A held-shuttle
4      guard it
5      builds candidate windows directly from each golden trajectory CSV under BOTH named
6      windowings
7      (:data:`backend.eval.windowing.PROPOSAL` and :data:`~backend.eval.windowing.SERVED`),
8      scores
9      CONTROL vs Gate-A (`net_crossings >= 2`) at each, and feeds the per-video held-out ?F1
10     into
11     :func:`backend.eval.promotion.promote_if_served_safe`.
12     It demonstrates the bug *and* the fix in one run:
13     - PROPOSAL windowing: Gate-A wins (historically **?F1 ? +0.074**, CI clears 0, 6/0
14     sign test).
15     - SERVED windowing: Gate-A does **not** win (historically **?F1 ? ?0.008**, real-
16     rally loss).
17     - The promotion guard therefore **vetoes** a promotion that would otherwise pass on
18     the
19     proposal numbers ? a cue cannot be promoted on a windowing production doesn't serve.
20
21     Unlike the persisted ``*_golden_features.json`` (which are PROPOSAL-windowed only), this
22     re-windows from the raw trajectory CSVs so the SERVED candidate set is genuinely
23     different.
24     Pure CPU, GPU-free, no Gemini, no DB. Reuses only shipped pieces (windowing, rally_gate,
25     metrics, promotion). Owner runs it against the real golden CSVs; tests exercise it on
26     synthetic
27     trajectories.
28

```

```

21     CLI::
22
23     python -m backend.eval.serve_contrast --manifest output/human_lovo_manifest.json
24     """
25     from __future__ import annotations
26
27     import argparse
28     import json
29     from typing import Any, Dict, List, Sequence, Tuple
30
31     from backend.eval import windowing as W
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.gemini_refine import merge_overlaps
34     from backend.eval.metrics import score
35     from backend.eval.promotion import promote_if_served_safe
36     from backend.pipeline.detectors import rally_gate
37     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
38
39     Interval = Tuple[float, float]
40     IOU = 0.5
41     MIN_CROSSINGS = 2
42
43
44     def gate_a_scores_at(points: List[Any], gts: List[Interval], fps: float, frame_width:
float,
45                          preset: W.WindowingPreset, iou: float = IOU,
46                          min_crossings: int = MIN_CROSSINGS, merge_gap: float = 1.0
47                          ) -> Tuple[float, float]:
48         """One video's (CONTROL F1, Gate-A F1) at ``preset``'s windowing.
49
50         Builds candidates under ``preset``, then scores two static variants vs ``gts``:
51         - CONTROL ? every candidate window, overlap-merged (the un-gated baseline).
52         - Gate-A ? keep only windows whose shuttle net-crossing count >=
``min_crossings``.
53
54         ``merge_gap`` is the post-keep overlap-merge gap (the harness's ``merge_overlaps``
default),
55         independent of the windowing preset's own coalescing gap. Returns the two F1
scores."""
56         windows = W.build_windows(points, fps, frame_width, preset)
57         intervals = W.windows_to_intervals(windows)
58         if not intervals:
59             return (0.0, 0.0)
60         est = rally_gate.estimate_play_axis(points)
61         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0,
estimate=est)
62         crossings = [g.crossings for g in gated]
63
64         control = merge_overlaps(intervals, gap_tol=merge_gap)
65         kept = [iv for iv, c in zip(intervals, crossings) if c >= min_crossings]
66         gate_a = merge_overlaps(kept, gap_tol=merge_gap)
67         return (score(control, gts, iou).f1, score(gate_a, gts, iou).f1)
68
69
70     def contrast_video(spec: Dict[str, Any], iou: float = IOU,
71                       min_crossings: int = MIN_CROSSINGS) -> Dict[str, Any]:
72         """Per-video CONTROL/Gate-A F1 at BOTH the proposal and served windowings."""
73         fps, fw = float(spec["fps"]), float(spec["frame_width"])
74         points = TrackNetRunner.parse_trajectory_csv(str(spec["trajectory"]))
75         gts = load_golden_gts(str(spec["labels"]))
76         p_control, p_gate = gate_a_scores_at(points, gts, fps, fw, W.PROPOSAL, iou,
min_crossings)
77         s_control, s_gate = gate_a_scores_at(points, gts, fps, fw, W.SERVED, iou,
min_crossings)
78         return {

```

```

79         "name": spec["name"],
80         "proposal": {"control_f1": p_control, "gate_a_f1": p_gate, "delta_f1": p_gate -
p_control},
81         "served": {"control_f1": s_control, "gate_a_f1": s_gate, "delta_f1": s_gate -
s_control},
82     }
83
84
85     def run_contrast(specs: Sequence[Dict[str, Any]], iou: float = IOU,
86                     min_crossings: int = MIN_CROSSINGS, min_n: int = 5,
87                     served_regress_eps: float = 0.0) -> Dict[str, Any]:
88         """Full corpus: per-video contrast + the served-guarded promotion decision for
Gate-A.
89
90         Gate-A is a *structural* guard (held-shuttle FP killer), so the promotion gate keeps
it ON
91         regardless of measured lift ? but it is only auto-promoted to the SERVED default
when it is
92         served-safe. This run shows it ISN'T (it regresses served), so the guard vetoes the
93         eval-only proposal win exactly as the foundation demands."""
94         per_video = [contrast_video(s, iou, min_crossings) for s in specs]
95         served_a = [v["served"]["control_f1"] for v in per_video]
96         served_b = [v["served"]["gate_a_f1"] for v in per_video]
97         proposal_a = [v["proposal"]["control_f1"] for v in per_video]
98         proposal_b = [v["proposal"]["gate_a_f1"] for v in per_video]
99
100        decision = promote_if_served_safe(
101            served_a, served_b,
102            proposal_a=proposal_a, proposal_b=proposal_b,
103            structural=True, min_n=min_n, served_regress_eps=served_regress_eps,
104        )
105        return {
106            "iou": iou,
107            "min_crossings": min_crossings,
108            "num_videos": len(per_video),
109            "per_video": per_video,
110            "decision": decision.as_dict(),
111        }
112
113
114    def format_contrast(result: Dict[str, Any]) -> str:
115        d = result["decision"]
116        prop, serv = d["proposal"], d["served"]
117        lines = [
118            f"=== Gate-A served-vs-proposal windowing contrast (n={result['num_videos']}, "
119            f"IoU>={result['iou']}, min_crossings={result['min_crossings']}) ===",
120            f"  {'video':<24}{'PROPOSAL ?F1':>14}{'SERVED ?F1':>14}",
121        ]
122        for v in result["per_video"]:
123            lines.append(f"  {'name':<24}{v['proposal']['delta_f1']:>+14.3f}"
124                        f"{'served':>+14.3f}")
125        lines += [
126            "  " + "-" * 50,
127            f"  PROPOSAL: mean ?F1 {prop['mean_delta']:+.4f} CI [{prop['ci_low']:+.4f}, "
128            f"{'ci_high':+.4f}] {int(prop['sign_test']['wins'])}W/"
129            f"{int(prop['sign_test']['losses'])}L regresses={prop['regresses']}",
130            f"  SERVED : mean ?F1 {serv['mean_delta']:+.4f} CI [{serv['ci_low']:+.4f}, "
131            f"{'ci_high':+.4f}] {int(serv['sign_test']['wins'])}W/"
132            f"{int(serv['sign_test']['losses'])}L regresses={serv['regresses']}",
133            "",
134            f"  promote={d['promote']} served_safe={d['served_safe']} "
135            f"vetoed_by_served={d['vetoed_by_served']}",
136            f"  -> {d['reason']}",
137        ]
138        return "\n".join(lines)

```

```

139
140
141 def main() -> None:
142     ap = argparse.ArgumentParser(
143         description="Gate-A eval?serve worked example: score CONTROL vs Gate-A at the
PROPOSAL "
144         "and SERVED windowings, then run the served-guarded promotion
gate.")
145     ap.add_argument("--manifest", default="output/human_lovo_manifest.json",
146                     help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
147     ap.add_argument("--iou", type=float, default=IOU)
148     ap.add_argument("--min-crossings", type=int, default=MIN_CROSSINGS)
149     ap.add_argument("--min-n", type=int, default=5, help="promotion min-N floor")
150     ap.add_argument("--served-regress-eps", type=float, default=0.0,
151                     help="served regression tolerance (0.0 = no served-mean drop
allowed)")
152     ap.add_argument("--json", default=None, help="also write the structured result JSON
here")
153     args = ap.parse_args()
154
155     with open(args.manifest, encoding="utf-8") as f:
156         specs = json.load(f)
157     result = run_contrast(specs, iou=args.iou, min_crossings=args.min_crossings,
158                          min_n=args.min_n, served_regress_eps=args.served_regress_eps)
159     print(format_contrast(result))
160     if args.json:
161         with open(args.json, "w", encoding="utf-8") as f:
162             json.dump(result, f, indent=2)
163             print(f"\nwrote {args.json}")
164
165
166 if __name__ == "__main__":
167     main()
168

```

7. user (2026-06-18T01:27:56.941Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10     filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11     active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12     -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21     python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple

```

```

28
29
30 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31 from backend.pipeline.detectors import rally_gate
32 from backend.eval.calibrate_wasb import load_golden_gts
33 from backend.eval.training import label_candidates
34 from backend.eval.classifier import LogisticRegression
35 from backend.eval.metrics import score
36 from backend.eval.gemini_refine import merge_overlaps
37 from backend.eval import windowing as _windowing
38
39 Interval = Tuple[float, float]
40
41 FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42 # Recall-oriented proposal: deliberately permissive so real rallies are among the
43 # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
44 # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
45 # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
46 # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
47 # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
48 PROPOSAL = _windowing.PROPOSAL.as_tuple()
49 BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
learning
50
51
52 def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
53                     proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
54     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
55     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
56     vt, mg, mwd = proposal
57     wins = TrackNetRunner.trajectory_to_action_windows(
58         points, fps=fps, frame_width=frame_width,
59         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
60     intervals = [(w["start"], w["end"]) for w in wins]
61     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
62     X: List[List[float]] = []
63     for w, g in zip(wins, gated):
64         dur = max(1e-6, w["end"] - w["start"])
65         win_frames = max(1.0, dur * fps)
66         X.append([
67             dur, # rally length (sec) ?
68             float(w["peak_velocity"]), # already normalized by
69             float(w["mean_velocity"]),
70             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
71             float(g.crossings), # net crossings (count) ?
72         ])
73     return intervals, X
74
75
76 def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
77     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
78     vt, mg, mwd = BASELINE_LOCAL
79     wins = TrackNetRunner.trajectory_to_action_windows(
80         points, fps=fps, frame_width=frame_width,

```

```

81         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82     return [(w["start"], w["end"]) for w in wins]
83
84
85     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86         fps, fw = float(spec["fps"]), float(spec["frame_width"])
87         intervals, X = build_candidates(spec["trajectory"], fps, fw)
88         gts = load_golden_gts(spec["labels"])
89         y = label_candidates(intervals, gts, iou)
90         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
91 X,
92         "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
93 fw)}
94
95     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
96 List[Interval],
97         threshold: float = 0.5) -> List[Interval]:
98         if not intervals:
99             return []
100         proba = model.predict_proba(X)
101         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
102         return merge_overlaps(pos, gap_tol=1.0)
103
104     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
105 model_out: Optional[str] = None) -> dict:
106         videos = [load_video(s, iou) for s in specs]
107         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
108 f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
109         for v in videos:
110             ceil = score(v["intervals"], v["gts"], iou).recall
111             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
112 f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
113
114         result: dict = {"videos": [v["name"] for v in videos]}
115         if len(videos) >= 2:
116             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
117 out) ---")
118             clf, base = [], []
119             for i, held in enumerate(videos):
120                 others = [v for j, v in enumerate(videos) if j != i]
121                 X = [row for v in others for row in v["X"]]
122                 y = [t for v in others for t in v["y"]]
123                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
124                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
125                 sc = score(preds, held["gts"], iou)
126                 bsc = score(held["baseline"], held["gts"], iou)
127                 clf.append(sc)
128                 base.append(bsc)
129                 print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
130 R={sc.recall:.3f} F1={sc.f1:.3f}"
131 f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
132 F1={bsc.f1:.3f}")
133             def mean(xs, a):
134                 return sum(getattr(s, a) for s in xs) / len(xs)
135             print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
136 R={mean(clf, 'recall'):.3f} "
137 f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
138 f"F1={mean(base, 'f1'):.3f} R={mean(base, 'recall'):.3f}")
139             result["mean_learned_f1"] = mean(clf, "f1")
140             result["mean_baseline_f1"] = mean(base, "f1")
141
142         # Final model on ALL videos (for production / running on a fresh held-out video).
143         Xall = [row for v in videos for row in v["X"]]

```

```

139     yall = [t for v in videos for t in v["y"]]
140     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141     assert final.w is not None # set by fit()
142     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143     print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144     if model_out:
145         final.save(model_out)
146         print(f"final model (all videos) -> {model_out}")
147         result["model_out"] = model_out
148     return result
149
150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154             help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166

```

8. assistant (2026-06-18T01:27:59.690Z)

Now let me read the remaining files: tracknet_runner.py (the windowing/trim/cap logic mentioned), rally_gate.py, and the audio module.

9. user (2026-06-18T01:28:00.765Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote

```



```

26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"           # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                        # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                       # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                       # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float

```

```

84         y: float
85
86
87     class TrackNetRunner(WslCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.
92         """
93
94         def __init__(self, cfg: TrackNetConfig):
95             self.cfg = cfg
96
97         def healthcheck(self) -> Tuple[bool, str]:
98             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100             Returns (ok, message). Cheap to call before a long run.
101             """
102             cmd = (
103                 f"conda activate {self.cfg.conda_env} && "
104                 f"python -c \"import tensorflow as tf; "
105                 f"print('TF', tf.__version__, 'GPUs', "
106                 f"len(tf.config.list_physical_devices('GPU')))\n"
107             )
108             try:
109                 res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "wsl` not found ? is this running on Windows with WSL2
112                 installed?"
113             except subprocess.TimeoutExpired:
114                 return False, "WSL healthcheck timed out."
115             out = (res.stdout or "").strip()
116             if res.returncode != 0:
117                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118             return True, out
119
120         # ----- inference
121
122         def run_predict(self, video_win_path: str, output_win_dir: str,
123             video_id: Optional[str] = None) -> Optional[str]:
124             """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125             None.
126
127             ``output_win_dir`` is a Windows directory (under the repo's output/) that
128             WSL writes into via its /mnt view, so the Windows process can read the CSV
129             back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130             therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131             a filename cannot collide on the output CSV.
132             """
133             if not self.cfg.weights_path:
134                 logger.error(
135                     "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136                     "in config.json to the WSL path of your trained TrackNetV4 weights."
137                 )
138                 return None
139
140             # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141             # relative paths through unchanged, so WSL would resolve them against the
142             # predict.py cwd instead of the repo (same bug class as wasb_runner).
143             output_win_dir = os.path.abspath(output_win_dir)
144             os.makedirs(output_win_dir, exist_ok=True)
145             # Normalize separators so basename/splitext work when tests (or collector)
146             # pass Windows-style paths on a Linux host.
147             video_win_path = str(video_win_path).replace("\\", "/")
148             base = os.path.basename(video_win_path)

```

```

146         stem, ext = os.path.splitext(base)
147
148         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149         if ext.lower() not in (".mp4", ".avi"):
150             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151             return None
152
153         wsl_out = to_wsl_mnt_path(output_win_dir)
154         # Namespace by video_id so two same-filename videos don't collide on the CSV.
155         # predict.py derives its output name from the (staged) video stem, so staging
156         # under the namespaced name propagates into "<key>_predict.csv".
157         key = f"{stem}_{video_id[:12]}" if video_id else stem
158         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160         # Resolve the video path WSL will read.
161         if self.cfg.stage_in_wsl:
162             staged = self._stage_video(video_win_path, f"{key}{ext}")
163             if staged is None:
164                 return None
165             wsl_video = staged
166         else:
167             # No staging: predict.py reads /mnt directly and writes
168             # "<stem>_predict.csv".
169             # We cannot rename its output, so fall back to the un-namespaced name.
170             wsl_video = to_wsl_mnt_path(video_win_path)
171             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173         if self.cfg.min_free_gb > 0:
174             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175             if free is not None and free < self.cfg.min_free_gb:
176                 logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                                "consider running tools/cleanup_caches.py.",
178                                free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180         cmd = (
181             f"conda activate {self.cfg.conda_env} && "
182             f"cd {self.cfg.repo_dir}/src && "
183             f"python predict.py "
184             f"--video_path {wsl_tilde_quote(wsl_video)} "
185             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186             f"--output_dir {wsl_tilde_quote(wsl_out)} "
187             f"--queue_length {self.cfg.queue_length}"
188         )
189         logger.info("Running TrackNet inference on %s ...", base)
190         try:
191             res = self._wsl_bash(cmd)
192         except subprocess.TimeoutExpired:
193             logger.error("TrackNet inference timed out after %ss.",
194                         self.cfg.timeout_sec)
195             return None
196
197         if res.returncode != 0:
198             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
199                     res.stdout)[-2000:])
200             return None
201
202         if not os.path.exists(expected_csv_win):
203             logger.error("TrackNet finished but expected CSV not found at %s",
204                         expected_csv_win)
205             return None
206
207         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
208
209         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
210         # regenerable intermediate). On earlier failure we return above without
211         cleaning.

```

```

206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
224         {wsl_tilde_quote(dest)} && echo OK"
225         try:
226             res = self._wsl_bash(cmd)
227         except subprocess.TimeoutExpired:
228             logger.error("Staging video into WSL timed out.")
229             return None
230         if res.returncode != 0 or "OK" not in (res.stdout or ""):
231             logger.error("Failed to stage video into WSL: %s", (res.stderr or
232 res.stdout).strip())
233             return None
234         return dest
235
236     # ----- CSV -> windows
237
238     @staticmethod
239     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
240         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
241         points: List[TrajectoryPoint] = []
242         with open(csv_win_path, newline="") as f:
243             reader = csv.DictReader(f)
244             for row in reader:
245                 try:
246                     points.append(TrajectoryPoint(
247                         frame=int(row["Frame"]),
248                         visible=int(row["Visibility"]) == 1,
249                         x=float(row["X"]),
250                         y=float(row["Y"]),
251                     ))
252                 except (KeyError, ValueError):
253                     continue
254             points.sort(key=lambda p: p.frame)
255             return points
256
257     @staticmethod
258     def trajectory_to_action_windows(
259         points: List[TrajectoryPoint],
260         fps: float,
261         frame_width: float,
262         chunk_start: float = 0.0,
263         velocity_thresh: float = 0.02,
264         merge_gap: float = 2.0,
265         min_window_duration: float = 2.0,
266         chunk_index: Optional[int] = None,
267         max_window_duration: float = 0.0,
268         min_split_gap: float = 0.5,
269         window_hard_cap: bool = False,
270         window_inactivity_end: float = 0.0,

```

```

269         inactivity_min_density: float = 0.34,
270         inactivity_rally_thresh: float = 0.08,
271         window_blob_fallback: bool = False,
272         window_blob_factor: float = 4.0,
273     ) -> List[Dict[str, Any]]:
274         """Convert a shuttle trajectory into candidate rally windows.
275
276         A frame is "active" when the shuttle is visible AND its inter-frame
277         displacement (normalised by frame width, per second) exceeds
278         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279         ``merge_gap`` seconds of each other are grouped into one window; short
280         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281         shape feeding the AI-handoff phase is identical.
282
283         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
284         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
285         progress" and a single window swallows several rallies + the dead time between them. When
286         set, any window longer than this is recursively SPLIT at its largest internal inactivity
287         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
288         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
289         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
291
292         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
293         an over-long window has NO internal inactivity gap to split on (a continuously-
294         active trajectory ? the shuttle never stops moving, e.g. the real-footage
295         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively
296         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
297         cuts (never merges), so recall cannot drop; gated default-OFF until measured.
298
299         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
300         unlike ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
301         every gap-less over-cap window and over-segments normal clips), this runs AFTER the R4
302         inactivity trim and force-chops ONLY a genuine *blob* ? a group still longer than
303         ``window_blob_factor`` x ``max_window_duration`` once the gap-split and the principled rally-end trim
304         have both had their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
305         is left alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
306         logged, and each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
307         that needs rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
308         (recall can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
309         measured.
310
311         """
312         if fps <= 0:
313             fps = 30.0
314         if frame_width <= 0:

```

```

313         frame_width = 1280.0
314
315     # Compute per-frame normalised velocity from the last *visible* point.
316     active = [] # (frame_idx, velocity)
317     prev: Optional[TrajectoryPoint] = None
318     for p in points:
319         if not p.visible:
320             prev = None # break the trajectory; absent shuttle = not in flight
321             continue
322         if prev is not None:
323             dframes = p.frame - prev.frame
324             if dframes > 0:
325                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                 velocity = (dist / frame_width) * (fps / dframes)
327                 if velocity > velocity_thresh:
328                     active.append((p.frame, velocity))
329             prev = p
330
331     if not active:
332         return []
333
334     max_gap_frames = int(merge_gap * fps)
335
336     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337         vels = [v for _, v in group]
338         return {
339             "start": group[0][0] / fps + chunk_start,
340             "end": group[-1][0] / fps + chunk_start,
341             "active_frame_count": len(group),
342             "peak_velocity": max(vels),
343             "mean_velocity": sum(vels) / len(vels),
344             "track_id_count": 1, # single shuttle; kept for window-shape parity
345             "chunk_index": chunk_index,
346         }
347
348     groups: List[List[Tuple[int, float]]] = []
349     group = [active[0]]
350     for af in active[1:]:
351         if af[0] - group[-1][0] <= max_gap_frames:
352             group.append(af)
353         else:
354             groups.append(group)
355             group = [af]
356     groups.append(group)
357
358     if max_window_duration > 0:
359         groups = TrackNetRunner._split_overlong_groups(
360             groups, int(max_window_duration * fps), int(min_split_gap * fps),
361             hard_cap=window_hard_cap)
362
363     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368     if window_inactivity_end > 0:
369         tw = int(window_inactivity_end * fps)
370         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                                                         inactivity_rally_thresh) for g
in groups]

```

```

372
373         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377         forced_flags = [False] * len(groups)
378         if window_blob_fallback and max_window_duration > 0:
379             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380                 groups, int(max_window_duration * fps), window_blob_factor)
381
382         windows = []
383         for g, forced in zip(groups, forced_flags):
384             w = _finalize(g)
385             if forced:
386                 w["forced_cut"] = True
387             windows.append(w)
388         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
393         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
394         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
395         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
396         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
397         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
398         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
399         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
400         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
401         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
402         if trail_frames <= 0 or len(group) < 2:
403             return group
404         frames = [f for f, _ in group]
405         fast = [1 if v > rally_thresh else 0 for _, v in group]
406         lo = 0
407         fast_sum = 0
408         last_dense = -1
409         for i in range(len(frames)):
410             fast_sum += fast[i]
411             while frames[lo] <= frames[i] - trail_frames:
412                 fast_sum -= fast[lo]
413                 lo += 1
414             win_count = i - lo + 1
415             if win_count > 0 and fast_sum / win_count >= min_density:
416                 last_dense = i
417         return group[: last_dense + 1] if last_dense >= 0 else group
418
419     @staticmethod
420     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                                 min_split_frames: int,
422                                 hard_cap: bool = False) -> List[List[Tuple[int, float]]]:

```

```

423         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at its
temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
428         true upper
429         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430         if max_win_frames <= 0:
431             return groups
432         out: List[List[Tuple[int, float]]] = []
433         work = list(groups)
434         while work:
435             g = work.pop()
436             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                 out.append(g)
438                 continue
439             best_i, best_gap = -1, -1
440             for i in range(1, len(g)):
441                 gap = g[i][0] - g[i - 1][0]
442                 if gap > best_gap:
443                     best_gap, best_i = gap, i
444             if best_i < 0 or best_gap < min_split_frames:
445                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447                 if hard_cap:
448                     mid = (g[0][0] + g[-1][0]) / 2.0
449                     split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450                     work.append(g[:split_i])
451                     work.append(g[split_i:])
452                 else:
453                     out.append(g)
454                     continue
455                 work.append(g[:best_i])
456                 work.append(g[best_i:])
457             out.sort(key=lambda grp: grp[0][0])
458         return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
466         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
467         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
469         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing

```



```

472 rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473 can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474 that piece came from a forced chop. Iterative worklist to bound stack depth."""
475 if max_win_frames <= 0:
476     return groups, [False] * len(groups)
477 blob_frames = max_win_frames * max(1.0, blob_factor)
478 out: List[List[Tuple[int, float]]] = []
479 flags: List[bool] = []
480 forced = 0
481 for g in groups:
482     if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483         out.append(g)
484         flags.append(False)
485         continue
486 pieces: List[List[Tuple[int, float]]] = []
487 work = [g]
488 while work:
489     h = work.pop()
490     if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491         pieces.append(h)
492         continue
493     mid = (h[0][0] + h[-1][0]) / 2.0
494     split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
495     work.append(h[:split_i])
496     work.append(h[split_i:])
497 pieces.sort(key=lambda grp: grp[0][0])
498 out.extend(pieces)
499 flags.extend([True] * len(pieces))
500 forced += len(pieces) - 1
501 if forced:
502     logger.warning(
503         "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504         "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505         "motion), not a bigger cap", forced)
506 return out, flags
507

```

10. assistant (2026-06-18T01:28:05.307Z)

Now let me look at the R5 blob fallback more carefully. I notice the `forced\_flags` aren't sorted with the groups when both are returned. Let me also read the rally_gate.py and audio module, plus check the merge_overlaps and related functions.

11. user (2026-06-18T01:28:06.765Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15

```

```

16 Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17 Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18 windows from trajectory_to_action_windows, or to fold into it later.
19 """
20 from __future__ import annotations
21
22 from dataclasses import dataclass
23 from typing import List, Optional, Sequence, Tuple
24
25 Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0

```

```

81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06,
93                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94         """Annotate each (start,end) window with its shuttle net-crossing count and a
95         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96         axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98         `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99         `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                   kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

12. user (2026-06-18T01:28:07.173Z)

```

backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
backend\eval\audio\__pycache__\__init__.cpython-313.pyc
backend\eval\audio\__pycache__\e1_probe.cpython-313.pyc

```

13. assistant (2026-06-18T01:28:11.035Z)

Let me focus on the R5 blob-fallback bug I noticed and look at the audio module. Let me examine the `\_blob\_fallback\_chop` more carefully ? the `forced\_flags` ordering issue and how it aligns with groups passed downstream.

14. user (2026-06-18T01:28:11.873Z)

```

1     """E1 ? audio GO/NO-GO probe (docs/AUDIO_RALLY_DETECTION_PLAN.md).
2
3     Answers, on REAL audio: does the shuttle "thwack" survive well enough that hit
4     peaks (a) cluster inside rallies far more than dead time, (b) recover rallies the
5     WASB windows miss (our recall bottleneck), and (c) form usable audio-only rally
6     proposals? No training, no fusion ? purely diagnostic, so we know whether to build
7     the rest of the audio path.
8

```

```

9 Reuses: audio.hit_detector, calibrate_wasb.load_golden_gts/make_predict_fn,
10 eval.metrics. Labels may be human-gold OR Gemini-silver ? silver is fine for the
11 SNR/efficacy question E1 asks.
12 """
13 from __future__ import annotations
14
15 import argparse
16 from typing import List, Tuple
17
18 from backend.pipeline.audio.hit_detector import detect_hits_in_video, HitEvent
19 from backend.eval.calibrate_wasb import load_golden_gts, make_predict_fn, BASELINE
20 from backend.eval.metrics import match_intervals, score
21 from backend.utils.video import get_video_duration
22
23 Interval = Tuple[float, float]
24
25
26 def cluster_hits(hits: List[HitEvent], gap_sec: float = 2.0, min_hits: int = 2,
27                 pad: float = 0.3) -> List[Interval]:
28     """Group hits whose inter-hit gap <= gap_sec into rally proposals; keep clusters
29     with >= min_hits (a lone hit = a service feed, not a rally)."""
30     ts = sorted(h.t for h in hits)
31     if not ts:
32         return []
33     out: List[Interval] = []
34     start = last = ts[0]
35     cnt = 1
36     for t in ts[1:]:
37         if t - last <= gap_sec:
38             last, cnt = t, cnt + 1
39         else:
40             if cnt >= min_hits:
41                 out.append((max(0.0, start - pad), last + pad))
42                 start = last = t
43                 cnt = 1
44             if cnt >= min_hits:
45                 out.append((max(0.0, start - pad), last + pad))
46     return out
47
48
49 def _hits_in(hits: List[HitEvent], s: float, e: float) -> int:
50     return sum(1 for h in hits if s <= h.t <= e)
51
52
53 def run(video: str, labels_csv: str, trajectory: str, fps: float, frame_width: float,
54        k: float = 2.5, cluster_gap: float = 2.0, min_hits: int = 2, iou: float = 0.5)
-> dict:
55     hits = detect_hits_in_video(video, k=k)
56     gts = load_golden_gts(labels_csv)
57     wasb = make_predict_fn(trajectory, fps, frame_width)(BASELINE)
58     dur = get_video_duration(video) or (max((e for _, e in gts + wasb), default=0.0))
59
60     print(f"=== E1 audio probe: {video} ===")
61     print(f" {len(hits)} hits detected (k={k}) over {dur:.0f}s =
{len(hits)/max(1,dur)*60:.1f} hits/min | "
62           f"{len(gts)} labeled rallies, {len(wasb)} WASB baseline windows")
63
64     # (1) Discrimination: hit density inside rallies vs dead time.
65     rally_dur = sum(e - s for s, e in gts)
66     dead_dur = max(1e-6, dur - rally_dur)
67     hits_in = sum(_hits_in(hits, s, e) for s, e in gts)
68     hits_out = len(hits) - hits_in
69     dens_in = hits_in / max(1e-6, rally_dur)
70     dens_out = hits_out / dead_dur
71     ratio = dens_in / dens_out if dens_out > 0 else float("inf")

```

```

72     rallies_with_hits = sum(1 for s, e in gts if _hits_in(hits, s, e) >= 2)
73     print(f" [discrimination] in-rally {dens_in:.2f} hits/s vs dead-time {dens_out:.2f}
hits/s "
74         f"? ratio {ratio:.1f}x | rallies with >=2 hits: {rallies_with_hits}/{len(gts)}
"
75         f"({100*rallies_with_hits/max(1, len(gts)):.0f}%)")
76
77     # (2) Recall recovery: of rallies WASB MISSES, how many have an audio hit-cluster?
78     mr = match_intervals(wasb, gts, iou)
79     missed = [gts[i] for i in mr.unmatched_gt_indices]
80     recoverable = [r for r in missed if _hits_in(hits, *r) >= min_hits]
81     rec_rate = len(recoverable) / len(missed) if missed else 0.0
82     print(f" [recall recovery] WASB misses {len(missed)}/{len(gts)} rallies; "
83         f"audio hit-clusters sit in {len(recoverable)} of them ? {100*rec_rate:.0f}%
recoverable")
84
85     # (3) Audio-only rally proposals vs labels.
86     props = cluster_hits(hits, cluster_gap, min_hits)
87     sc = score(props, gts, iou)
88     print(f" [audio-only proposals] {len(props)} rallies | P={sc.precision:.3f}
R={sc.recall:.3f} "
89         f"F1={sc.f1:.3f} (vs WASB-only F1 {score(wasb, gts, iou).f1:.3f})")
90     return {
91         "video": video, "k": k, "n_hits": len(hits), "duration": dur,
92         "discrimination_ratio": ratio, "rallies_with_hits_pct": 100 * rallies_with_hits
/ max(1, len(gts)),
93         "wasb_missed": len(missed), "recall_recoverable_pct": 100 * rec_rate,
94         "audio_only": sc.as_dict(), "wasb_only_f1": score(wasb, gts, iou).f1,
95     }
96
97
98     def main() -> None:
99         ap = argparse.ArgumentParser(description="E1 audio go/no-go probe.")
100         ap.add_argument("--video", required=True)
101         ap.add_argument("--labels", required=True, help="golden or Gemini-silver rally CSV")
102         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV")
103         ap.add_argument("--fps", type=float, default=59.94)
104         ap.add_argument("--frame-width", type=float, default=1920.0)
105         ap.add_argument("--k", type=float, default=2.5, help="onset sensitivity (lower =
more hits)")
106         ap.add_argument("--cluster-gap", type=float, default=2.0)
107         ap.add_argument("--min-hits", type=int, default=2)
108         ap.add_argument("--iou", type=float, default=0.5)
109         args = ap.parse_args()
110         run(args.video, args.labels, args.trajectory, args.fps, args.frame_width,
111             k=args.k, cluster_gap=args.cluster_gap, min_hits=args.min_hits, iou=args.iou)
112
113
114     if __name__ == "__main__":
115         main()
116

```

15. assistant (2026-06-18T01:28:18.526Z)

Now let me carefully analyze the R5 blob-fallback ordering issue. In ``trajectory_to_action_windows``, after ``_blob_fallback_chop``, ``groups`` and ``forced_flags`` are zipped together. But note ``_blob_fallback_chop`` does NOT re-sort ``out`` globally ? it preserves group order from the input but the chopped pieces are sorted within each group. The key question: are groups still sorted after R4 trim and is ``_finalize`` using ``group[0][0]`` and ``group[-1][0]`` ?

Let me look more carefully at a subtle issue. In ``_split_overlong_groups``, the output IS sorted (``out.sort``). But ``_trim_inactive_tail`` is applied per-group via list comprehension, preserving order. Then ``_blob_fallback_chop`` preserves group order. So ordering downstream looks fine.

Let me check the `\_split\_overlong\_groups` hard_cap midpoint logic for a potential infinite loop, and look at the test files to understand contracts.

16. user (2026-06-18T01:28:19.243Z)

```
tests\test_fusion_golden_telemetry.py
tests\__pycache__\test_fusion_golden_telemetry.cpython-313-pytest-9.0.3.pyc
tests\test_golden_fixtures.py
tests\test_golden_real_fixtures.py
```

17. user (2026-06-18T01:28:19.758Z)

```
tests\__pycache__\test_tracknet_hybrid.cpython-313.pyc
tests\__pycache__\test_tracknet_runner.cpython-313.pyc
tests\__pycache__\test_tracknet_hybrid.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_tracknet_runner.cpython-313-pytest-9.0.3.pyc
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
```

18. user (2026-06-18T01:28:19.826Z)

No files found

19. user (2026-06-18T01:28:26.692Z)

No files found

20. user (2026-06-18T01:28:27.253Z)

No files found

21. user (2026-06-18T01:28:31.553Z)

```
Found 7 files
tests\test_windowing.py
tests\test_tracknet_runner.py
docs\QUALITY_ITERATIONS.md
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\detectors\tracknet_runner.py
backend\eval\windowing.py
backend\config\models.py
```

22. assistant (2026-06-18T01:28:36.002Z)

Let me look at how these knobs are used by `windowing.py` and the segmenter, and check the tests for the cap/blob logic.

23. user (2026-06-18T01:28:37.316Z)

```
1      """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3      **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4      ?
5      ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
6      built
7      on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8      deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9      pipeline
10     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11     merge_gap=2.0,
12     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13     is
14     measured on candidates **production never serves**, so a "win" need not transfer. This
```

```

is
10     exactly how Gate-A scored **+0.074** on the proposal set yet **20.008** on the served
set
11     (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).
12
13     **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14     SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17     *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18     operating point a given deployment truly serves.
19
20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21     windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22     ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float

```

```

61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66         hard_cap: bool = False
67         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69         inactivity_end: float = 0.0
70         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71         inactivity_rally_thresh: float = 0.08
72         inactivity_min_density: float = 0.34
73         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76         #: factor keeps it surgical (a normal long rally is left alone).
77         blob_fallback: bool = False
78         blob_factor: float = 4.0
79
80         def as_tuple(self) -> Tuple[float, float, float]:
81             """(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83             bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86         def to_dict(self) -> Dict[str, Any]:
87             return {
88                 "name": self.name,
89                 "velocity_thresh": self.velocity_thresh,
90                 "merge_gap": self.merge_gap,
91                 "min_window_duration": self.min_window_duration,
92                 "max_window_duration": self.max_window_duration,
93                 "hard_cap": self.hard_cap,
94                 "inactivity_end": self.inactivity_end,
95                 "inactivity_rally_thresh": self.inactivity_rally_thresh,
96                 "inactivity_min_density": self.inactivity_min_density,
97                 "blob_fallback": self.blob_fallback,
98                 "blob_factor": self.blob_factor,
99             }
100
101
102         def _served_preset_from_defaults() -> WindowingPreset:
103             """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105             ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106             - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107             - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108             - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110             Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is

```



```

the whole
111     point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112     so the two can never silently diverge again.
113     """
114     idx = IndexingConfig() # construct with defaults ? the served operating point
115     return WindowingPreset(
116         name="served",
117         velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
118         merge_gap=float(idx.dead_time_merge_gap),
119         min_window_duration=float(idx.min_action_window_duration),
120         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121         hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122         inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",

```

```

SERVED.velocity_thresh)),
160         merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161         min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162         max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163         hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164         inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165         inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166         inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167         blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168         blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169     )
170
171
172 def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173     """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
174     (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175     if obj is None:
176         return {}
177     if isinstance(obj, Mapping):
178         sub = obj.get(key, {})
179     elif hasattr(obj, "get"):
180         sub = obj.get(key, {})
181     else:
182         sub = getattr(obj, key, {})
183     return sub if isinstance(sub, Mapping) else {}
184
185
186 def build_windows(points: List[Any], fps: float, frame_width: float,
187                 preset: WindowingPreset) -> List[Dict[str, Any]]:
188     """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
189
190     A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
191     instead of three loose floats, so eval candidate-building and the served path
reference the
192     SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195     vt, mg, mwd = preset.as_tuple()
196     return TrackNetRunner.trajectory_to_action_windows(
197         points, fps=fps, frame_width=frame_width,
198         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199         max_window_duration=preset.max_window_duration,
200         window_hard_cap=preset.hard_cap,
201         window_inactivity_end=preset.inactivity_end,
202         inactivity_rally_thresh=preset.inactivity_rally_thresh,
203         inactivity_min_density=preset.inactivity_min_density,
204         window_blob_fallback=preset.blob_fallback,
205         window_blob_factor=preset.blob_factor)
206
207
208 def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209     """`[{start, end, ...}]` ? `[ (start, end) ]` ? the interval shape the scorer/gate
speak."""
210     return [(w["start"], w["end"]) for w in windows]
211

```

24. assistant (2026-06-18T01:28:44.687Z)

Now let me carefully re-examine the interaction between R4 trim and R5 blob-fallback ordering. There's a subtle issue I want to verify. The ``min_window_duration`` filter at the end (line 388) is applied AFTER R4 trim and R5 chop. Let me trace a potential bug:

In ``trajectory_to_action_windows``, after ``_blob_fallback_chop`` returns ``(groups, forced_flags)``, the code does:

```
```python
for g, forced in zip(groups, forced_flags):
 w = _finalize(g)
 if forced:
 w["forced_cut"] = True
 windows.append(w)
return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
```
```

The ``min_window_duration`` filter runs at the very end. Now let me look at a potential issue with R4 trim producing an empty group. ``_trim_inactive_tail`` can return a sliced group ``group[:last_dense+1]``. If ``last_dense`` is 0, it returns a single-element group ``[group[0]]``. Then ``_finalize`` on a single-element group: ``start = group[0][0]/fps``, ``end = group[-1][0]/fps`` ? same value, so ``end - start = 0``, filtered out by ``min_window_duration``. That's fine.

Let me now focus on the test for `tracknet_runner` to understand expected behavior, and check ``_split_overlong_groups`` hard_cap termination edge cases.

25. user (2026-06-18T01:28:45.636Z)

```
1      """Tests for the TrackNet runner's pure-Python pieces (path translation, CSV
2      parsing, trajectory -> action-window conversion). The WSL subprocess invocation
3      itself is not exercised here ? these run anywhere, no WSL/TensorFlow needed."""
4
5      import os
6      import subprocess
7      import textwrap
8      from unittest.mock import patch
9
10     from backend.pipeline.detectors.tracknet_runner import (
11         TrackNetRunner,
12         TrackNetConfig,
13         TrajectoryPoint,
14         to_wsl_mnt_path,
15     )
16     from backend.pipeline.detectors.base import DetectorRunner
17     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
18
19
20     def _proc(returncode=0, stdout="", stderr=""):
21         return subprocess.CompletedProcess(args=[], returncode=returncode,
22             stdout=stdout, stderr=stderr)
23
24
25     def test_to_wsl_mnt_path_windows():
26         assert to_wsl_mnt_path(r"C:\Users\avidu\v.mp4") == "/mnt/c/Users/avidu/v.mp4"
27         assert to_wsl_mnt_path(r"D:\clips\m.mp4") == "/mnt/d/clips/m.mp4"
28
29
30     def test_to_wsl_mnt_path_passthrough():
31         # Already-POSIX paths are left alone.
32         assert to_wsl_mnt_path("/home/avidullu/clips/v.mp4") == "/home/avidullu/clips/v.mp4"
33         assert to_wsl_mnt_path("~/clips/v.mp4") == "~/clips/v.mp4"
34
35
36     def test_config_from_indexing_cfg():
37         cfg = TrackNetConfig.from_indexing_cfg({"tracknet": {
```

```

38         "wsl_distro": "Ubuntu-22.04", "conda_env": "TN", "weights_path": "/w/m.h5",
39         "queue_length": 7, "stage_in_wsl": False,
40     })
41     assert cfg.distro == "Ubuntu-22.04"
42     assert cfg.conda_env == "TN"
43     assert cfg.weights_path == "/w/m.h5"
44     assert cfg.queue_length == 7
45     assert cfg.stage_in_wsl is False
46
47
48     def test_config_defaults_when_absent():
49         cfg = TrackNetConfig.from_indexing_cfg({})
50         assert cfg.distro == "Ubuntu"
51         assert cfg.conda_env == "TrackNetV4"
52         assert cfg.weights_path == "" # must be supplied by the user
53
54
55     def test_parse_trajectory_csv(tmp_path):
56         csv_file = tmp_path / "v_predict.csv"
57         csv_file.write_text(textwrap.dedent("""\
58             Frame,Visibility,X,Y
59             0,0,0,0
60             1,1,100,200
61             2,1,140,210
62             3,0,0,0
63         """))
64         pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
65         assert len(pts) == 4
66         assert pts[0].visible is False
67         assert pts[1].visible is True and pts[1].x == 100 and pts[1].y == 200
68         assert pts[3].visible is False
69
70
71     def test_trajectory_to_action_windows_detects_moving_shuttle():
72         # 30 fps, 1280px wide. Shuttle moves ~60px/frame while visible (frames 0..29) =>
73         # well above the velocity threshold; then disappears. Expect one window ~1s long.
74         fps, width = 30.0, 1280.0
75         pts = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
76 range(30)]
77         pts += [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(30, 60)]
78
79         windows = TrackNetRunner.trajectory_to_action_windows(
80             pts, fps=fps, frame_width=width, velocity_thresh=0.02,
81             merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
82         )
83         assert len(windows) == 1
84         w = windows[0]
85         assert w["start"] < w["end"]
86         assert w["active_frame_count"] > 0
87         assert w["peak_velocity"] > 0.02
88         assert w["chunk_index"] == 0
89
90     def test_trajectory_to_action_windows_ignores_stationary_shuttle():
91         # Visible but barely moving (1px/frame) => below threshold => no windows.
92         pts = [TrajectoryPoint(frame=i, visible=True, x=500 + i, y=300) for i in range(60)]
93         windows = TrackNetRunner.trajectory_to_action_windows(
94             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.05,
95             merge_gap=2.0, min_window_duration=0.5,
96         )
97         assert windows == []
98
99
100     def test_trajectory_to_action_windows_splits_on_gap():
101         # Two bursts of motion separated by a long invisible gap => two windows.

```

```

102     burst1 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
range(20)]
103     gap = [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(20, 140)]
104     burst2 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - 140), y=300)
105               for i in range(140, 160)]
106     windows = TrackNetRunner.trajectory_to_action_windows(
107         burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
108         velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
109     )
110     assert len(windows) == 2
111     assert windows[0]["end"] < windows[1]["start"]
112
113
114     # ---- W1: bounded windowing (max_window_duration split)
----- #
115     def _moving(a, b): # visible, fast-moving shuttle over frames [a, b)
116         return [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - a), y=300) for i in
range(a, b)]
117
118
119     def _invisible(a, b):
120         return [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(a, b)]
121
122
123     # Two rallies merged across a sub-merge_gap dead time ? ONE over-long window without the
cap.
124     _MERGED_PTS = _moving(0, 121) + _invisible(121, 162) + _moving(162, 243) # ~8s, 1.4s
internal gap
125
126
127     def test_max_window_duration_off_keeps_over_merge():
128         # Default (max_window_duration=0): the two rallies stay merged into one window.
129         w = TrackNetRunner.trajectory_to_action_windows(
130             _MERGED_PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
131             merge_gap=2.0, min_window_duration=0.3)
132         assert len(w) == 1 # over-merge reproduced (the failure the cap fixes)
133         assert (w[0]["end"] - w[0]["start"]) > 7.0
134
135
136     def test_max_window_duration_splits_at_inactivity_gap():
137         # With a 4s cap the 8s merged window is split at its 1.4s internal dead-time gap ? 2
windows.
138         w = TrackNetRunner.trajectory_to_action_windows(
139             _MERGED_PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
140             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
min_split_gap=0.5)
141         assert len(w) == 2
142         assert w[0]["end"] < w[1]["start"]
143         assert all((x["end"] - x["start"]) <= 5.0 for x in w)
144
145
146     def test_max_window_duration_preserves_genuine_long_rally():
147         # A single continuous 10s rally (no internal gap ? min_split_gap) is NOT split, even
148         # though it exceeds the cap ? bounded windowing only cuts at real dead time.
149         w = TrackNetRunner.trajectory_to_action_windows(
150             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
151             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
min_split_gap=0.5)
152         assert len(w) == 1
153         assert (w[0]["end"] - w[0]["start"]) > 9.0
154
155
156     def test_split_overlong_groups_pure():
157         # Direct check of the splitter: a group with one big internal gap splits there.
158         grp = [(i, 1.0) for i in range(0, 20)] + [(i, 1.0) for i in range(60, 80)] # gap

```

```

20..60
159         out = TrackNetRunner._split_overlong_groups([grp], max_win_frames=30,
min_split_frames=10)
160         assert len(out) == 2 and out[0][-1][0] == 19 and out[1][0][0] == 60
161         # No qualifying gap (all consecutive) ? unchanged even when over length.
162         solid = [(i, 1.0) for i in range(0, 100)]
163         assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
164
165
166     # ---- W1 hard-cap fallback: enforce the cap on continuously-active trajectories
----- #
167     def test_window_hard_cap_chops_continuous_rally():
168         # The same continuous ~10s rally that bounded-windowing alone LEAVES intact (no
internal
169         # gap) is, with window_hard_cap=True, chopped at the midpoint until every piece ?
the cap.
170         w = TrackNetRunner.trajectory_to_action_windows(
171             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
172             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
min_split_gap=0.5,
173             window_hard_cap=True)
174         assert len(w) >= 3                                # the 10s blob is broken
up
175         assert all((x["end"] - x["start"]) <= 4.01 for x in w)    # every piece within the
cap
176         assert w[0]["start"] <= 0.1 and w[-1]["end"] >= 9.0    # coverage preserved
(only cuts)
177
178
179     def test_split_overlong_groups_hard_cap_pure():
180         solid = [(i, 1.0) for i in range(0, 100)]            # 99-frame span, no
internal gap
181         # default: kept whole; hard_cap: chopped into pieces each spanning ? max_win_frames
182         assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
183         out = TrackNetRunner._split_overlong_groups([solid], 30, 10, hard_cap=True)
184         assert len(out) >= 4
185         assert all((g[-1][0] - g[0][0]) <= 30 for g in out)
186         # only cuts, never drops/duplicates: the pieces partition the original frames
exactly
187         assert sorted(f for g in out for f, _ in g) == list(range(0, 100))
188
189
190     # ---- R4: rally-END inactivity trim (drop the post-rally slow-drift tail)
----- #
191     def test_trim_inactive_tail_drops_slow_drift():
192         # fast rally (vel 0.5) frames 0..60, then slow drift (vel 0.03, still > active 0.02)
61..200.
193         group = [(i, 0.5) for i in range(0, 61)] + [(i, 0.03) for i in range(61, 201)]
194         out = TrackNetRunner._trim_inactive_tail(group, trail_frames=30, min_density=0.3,
rally_thresh=0.2)
195         assert out[0][0] == 0                                # start untouched
196         assert out[-1][0] < 100                            # the pure-drift tail (frames
100..200) is trimmed off
197         assert len(group) - len(out) >= 100                # ~all the slow drift cut (kept only a
short decay tail)
198
199
200     def test_trim_inactive_tail_keeps_continuous_rally_and_is_noop_when_off():
201         allfast = [(i, 0.5) for i in range(0, 100)]
202         assert TrackNetRunner._trim_inactive_tail(allfast, 30, 0.3, 0.2) == allfast    #
fully-fast: untouched
203         assert TrackNetRunner._trim_inactive_tail(allfast, 0, 0.3, 0.2) == allfast    #
trail=0: OFF/no-op
204
205

```

```

206     def test_window_inactivity_end_trims_through_api():
207         # one window of fast motion then slow drift (both visible & > velocity_thresh ? one
group).
208         fast = [TrajectoryPoint(frame=i, visible=True, x=100 + 40 * (i % 5), y=300) for i in
range(0, 90)]
209         slow = [TrajectoryPoint(frame=i, visible=True, x=100 + (i % 2), y=300) for i in
range(90, 300)]
210         pts = fast + slow
211         off = TrackNetRunner.trajectory_to_action_windows(
212             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3)
213         on = TrackNetRunner.trajectory_to_action_windows(
214             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3,
215             window_inactivity_end=1.0, inactivity_rally_thresh=0.2,
inactivity_min_density=0.3)
216         assert on and off
217         assert on[0]["end"] < off[0]["end"]          # R4 trims the drift tail ? earlier END
218         assert on[0]["start"] == off[0]["start"]      # START unchanged
219
220
221     # ---- R5: blob-fallback (force-split the continuously-active blob, default OFF)
----- #
222     def test_blob_fallback_chop_pure():
223         blob = [(i, 1.0) for i in range(0, 200)]      # 199-frame span, no gap = 6.6× a
30-frame cap
224         out, flags = TrackNetRunner._blob_fallback_chop([blob], max_win_frames=30,
blob_factor=4.0)
225         assert len(out) >= 4 and all(flags)           # force-chopped, every
piece flagged
226         assert all((g[-1][0] - g[0][0]) <= 30 for g in out)      # every piece within
the cap
227         assert sorted(f for g in out for f, _ in g) == list(range(0, 200)) # partition
(only cuts)
228
229
230     def test_blob_fallback_factor_leaves_normal_long_rally_alone():
231         # 79-frame span = 2.6× a 30-frame cap ? BELOW the 4× blob threshold ? left ALONE
(the gate that
232         # keeps R5 surgical; this is the over-cap-but-not-a-blob case hard_cap would wrongly
chop).
233         normal = [(i, 1.0) for i in range(0, 80)]
234         out, flags = TrackNetRunner._blob_fallback_chop([normal], max_win_frames=30,
blob_factor=4.0)
235         assert out == [normal] and flags == [False]
236
237
238     def test_window_blob_fallback_rescues_blob_through_api():
239         # A continuous ~10s blob (no gap) the cap alone leaves whole: blob_fallback breaks
it up + flags.
240         pts = _moving(0, 301)                          # 10s continuous, no
internal gap
241         off = TrackNetRunner.trajectory_to_action_windows(
242             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3, max_window_duration=2.0)      # cap 2s, blob_fallback
OFF
243
244         on = TrackNetRunner.trajectory_to_action_windows(
245             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3, max_window_duration=2.0,
246             window_blob_fallback=True, window_blob_factor=4.0)    # 10s > 4×2s threshold ?
a blob
247
248         assert len(off) == 1 and (off[0]["end"] - off[0]["start"]) > 9.0    # cap alone
leaves it whole
249         assert len(on) >= 4          # the blob is broken up
250         assert all((w["end"] - w["start"]) <= 2.01 for w in on)    # every piece within the

```

```

cap
251         assert all(w.get("forced_cut") for w in on)                # every piece flagged a
forced cut
252
253
254     def test_window_blob_fallback_noop_without_cap():
255         # blob_fallback requires a cap to define "over-cap"; with max_window_duration=0 it's
a no-op.
256         w = TrackNetRunner.trajectory_to_action_windows(
257             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
merge_gap=2.0,
258             min_window_duration=0.3, window_blob_fallback=True)
259         assert len(w) == 1 and not w[0].get("forced_cut")
260
261
262     # ----- WSL subprocess glue
263
264     def _runner():
265         return TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5"))
266
267
268     def test_healthcheck_ok():
269         r = _runner()
270         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF 2.17.0 GPUs 1")):
271             ok, msg = r.healthcheck()
272             assert ok is True
273             assert "TF 2.17.0" in msg
274
275
276     def test_healthcheck_failure_returns_message():
277         r = _runner()
278         with patch.object(r, "_wsl_bash", return_value=_proc(1,
stderr="ModuleNotFoundError")):
279             ok, msg = r.healthcheck()
280             assert ok is False
281             assert "ModuleNotFoundError" in msg
282
283
284     def test_healthcheck_no_wsl_binary():
285         r = _runner()
286         with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
287             ok, msg = r.healthcheck()
288             assert ok is False
289             assert "wsl" in msg.lower()
290
291
292     def test_run_predict_requires_weights():
293         r = TrackNetRunner(TrackNetConfig(weights_path="")) # no weights configured
294         assert r.run_predict(r"C:\v\clip.mp4", "out") is None
295
296
297     def test_run_predict_rejects_bad_extension(tmp_path):
298         r = _runner()
299         assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is None
300
301
302     def test_run_predict_success(tmp_path):
303         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
304         out_dir = str(tmp_path)
305         # predict.py would write "<stem>_predict.csv"; create it so the existence check
passes.
306         csv_path = os.path.join(out_dir, "clip_predict.csv")
307         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="Prediction
complete.)):
308             open(csv_path, "w").close()

```



```

309         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
310     assert result == csv_path
311
312
313 def test_run_predict_failure_returns_none(tmp_path):
314     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
315     with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="CUDA error")):
316         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
317
318
319 def test_run_predict_missing_output_csv(tmp_path):
320     """Exit 0 but no CSV produced should be treated as failure."""
321     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
322     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
323         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
324
325
326 def test_stage_video_success():
327     r = _runner()
328     # Two WSL calls now: the readability preflight (echo READABLE), then the cp (echo
329     OK).
330     with patch.object(r, "_wsl_bash",
331                       side_effect=[_proc(0, stdout="READABLE"), _proc(0, stdout="OK")]):
332         staged = r._stage_video(r"C:\v\clip.mp4", "clip.mp4")
333         assert staged == "~/clips/clip.mp4"
334
335 def test_stage_video_failure():
336     r = _runner()
337     # Source readable, but the cp itself fails -> None.
338     with patch.object(r, "_wsl_bash",
339                       side_effect=[_proc(0, stdout="READABLE"), _proc(1, stderr="disk
340 full)"]):
341         assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None
342
343 def test_stage_video_unreadable_source_fails_fast_with_actionable_msg(caplog):
344     """A source invisible to WSL (Google Drive / UNC) must fail BEFORE cp, with a clear,
345     fixable message ? not a cryptic 'cannot stat' that silently yields 0 rallies."""
346     r = _runner()
347     import logging
348     # Preflight `test -r` returns no READABLE -> unreadable. cp must NOT be attempted.
349     with patch.object(r, "_wsl_bash", return_value=_proc(1, stdout="")) as wsl, \
350         caplog.at_level(logging.ERROR):
351         assert r._stage_video("G:\\My Drive\\Sharing\\clip.mp4", "clip.mp4") is None
352     assert wsl.call_count == 1 # only the preflight ran; cp short-circuited
353     assert "not readable from WSL" in caplog.text
354     assert "local disk" in caplog.text
355
356
357 def test_wsl_source_unreadable_reason_none_when_readable():
358     r = _runner()
359     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="READABLE")):
360         assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
361
362
363 def test_wsl_source_unreadable_reason_none_on_flaky_probe():
364     """A timed-out/raising probe must NOT block a genuinely-readable source."""
365     r = _runner()
366     with patch.object(r, "_wsl_bash", side_effect=subprocess.TimeoutExpired(cmd="x",
367     timeout=1)):
368         assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
369
370
371 def test_run_predict_stages_when_enabled(tmp_path):

```

```

371     """With stage_in_wsl=True, run_predict should stage the video first."""
372     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=True))
373     out_dir = str(tmp_path)
374     csv_path = os.path.join(out_dir, "clip_predict.csv")
375     with patch.object(r, "_stage_video", return_value="-/clips/clip.mp4") as mock_stage,
376     \
377         patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
378         open(csv_path, "w").close()
379         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
380     mock_stage.assert_called_once()
381     assert result == csv_path
382
383     # ----- DetectorRunner ABC
384     contract
385     def test_tracknet_runner_implements_detector_runner_abc():
386         """TrackNetRunner must be a DetectorRunner (for polymorphism / future native
387         runners)."""
388         r = _runner()
389         assert isinstance(r, DetectorRunner)
390         # healthcheck contract: (bool, str) ? exercised with patch in other tests, but shape
391         here too
392         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF ok")):
393             res = r.healthcheck()
394             assert isinstance(res, tuple) and len(res) == 2
395             assert isinstance(res[0], bool)
396             assert isinstance(res[1], str)
397
398     def test_wasb_runner_implements_detector_runner_abc():
399         """WasbRunner must also satisfy the DetectorRunner contract."""
400         w = WasbRunner(WasbConfig())
401         assert isinstance(w, DetectorRunner)
402         with patch.object(w, "_wsl_bash", return_value=_proc(0, stdout="torch ok")):
403             res = w.healthcheck()
404             assert isinstance(res, tuple) and len(res) == 2
405             assert isinstance(res[0], bool) and isinstance(res[1], str)
406
407     # ----- WASB streaming fast-path wiring
408
409     def test_wasb_config_stream_video_default_off_and_parsing():
410         """Streaming is OFF by default (production behavior unchanged) and opt-in via
411         cfg."""
412         assert WasbConfig().stream_video is False
413         cfg = WasbConfig.from_indexing_cfg({"wasb": {"stream_video": True}})
414         assert cfg.stream_video is True
415         assert WasbConfig.from_indexing_cfg({"wasb": {}}).stream_video is False
416
417     def _wasb_run_predict_cmd(stream_video, tmp_path):
418         """Drive WasbRunner.run_predict with WSL fully mocked and return the inference cmd
419         string (the last _wsl_bash call carries `python wasb_infer.py ...`)."""
420         r = WasbRunner(WasbConfig(stream_video=stream_video))
421         out_dir = str(tmp_path)
422         key = "clip__abc123abc123"
423         csv_path = os.path.join(out_dir, f"{key}_wasb.csv")
424         with patch.object(r, "_stage_infer_script", return_value=True), \
425             patch.object(r, "_stage_video", return_value="-/clips/clip.mp4"), \
426             patch.object(r, "_wsl_rm_rf"), \
427             patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")) as wsl:
428             open(csv_path, "w").close()
429             result = r.run_predict(r"C:\v\clip.mp4", out_dir, video_id="abc123abc123def")
430         assert result == csv_path

```

```

431         # The inference invocation is the call whose command contains wasb_infer.py.
432         infer_cmds = [c.args[0] for c in wsl.call_args_list if "wasb_infer.py" in c.args[0]]
433         assert len(infer_cmds) == 1
434         return infer_cmds[0]
435
436
437     def test_run_predict_streaming_passes_stream_flag_not_frames_dir(tmp_path):
438         cmd = _wasb_run_predict_cmd(stream_video=True, tmp_path=tmp_path)
439         assert "--stream-video" in cmd
440         assert "--frames_out_dir" not in cmd      # no PNG extraction on the fast path
441         assert "--video " in cmd                  # still decodes the staged video
442
443
444     def test_run_predict_default_passes_frames_dir_not_stream_flag(tmp_path):
445         cmd = _wasb_run_predict_cmd(stream_video=False, tmp_path=tmp_path)
446         assert "--stream-video" not in cmd        # default path unchanged
447         assert "--frames_out_dir" in cmd
448
449
450     # (Intentionally no "no-wsl unpatched predict" test here: the stage/inference paths
451     # raise FileNotFoundError on this Linux box when "wsl" is absent, while only the
452     # healthcheck paths catch it today. The graceful-None cases are already exercised
453     # by the patched tests above and by the hybrid tests. Adding a broad except in
454     # run_predict would be a separate robustness change.)
455

```

26. assistant (2026-06-18T01:28:49.359Z)

Now I have a good understanding. Let me look at the `gemini\_refine.merge\_overlaps` and `metrics.score` functions since they're used by the replay and serve_contrast, to verify the contract assumptions. Let me also check the `count\_net\_crossings` deadband and the `\_points\_in` frame-comparison since that could have an off-by-one.

27. user (2026-06-18T01:28:50.759Z)

```

1     """Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).
2
3     WASB (shuttle-only) over-fires on between-point service feeds. This drives the
4     EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
5     each candidate window is cut to a small padded clip and handed to Gemini, which ?
6     seeing the actual footage ? returns the true rally inside it (tight boundaries) or
7     nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
8     not GB-scale chunks; high-precision because Gemini judges each candidate visually.
9
10    Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
11    sports.badminton.prompt (sport_registry), utils.video.extract_video_chunk,
12    calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.
13
14    This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
15    trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.
16
17    Run (GEMINI_API_KEY in env):
18        python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv
19    \
20        --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
21    """
22    from __future__ import annotations
23
24    import os
25    import os.path as osp
26    import time
27    import argparse
28    import concurrent.futures
29    from typing import List, Tuple, Optional

```

```

30 from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
31 from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
32 from backend.utils.video import get_video_duration, extract_video_chunk
33 from backend.sports import sport_registry
34 from backend.providers import provider_registry
35
36 Interval = Tuple[float, float]
37
38
39 def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
40     """Union of overlapping intervals, optionally stitching across a small gap.
41
42     gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
43     shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
44     (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
45     which over-merged multiple real rallies WASB had bundled into one candidate.)"""
46     merged: List[Interval] = []
47     for s, e in sorted(intervals):
48         if merged and s <= merged[-1][1] + gap_tol:
49             merged[-1] = (merged[-1][0], max(merged[-1][1], e))
50         else:
51             merged.append((s, e))
52     return merged
53
54
55 def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
56     """Map a Gemini clip's returned segments back to full-video seconds, clamped to
57     clip."""
58     out: List[Interval] = []
59     for sg in raw or []:
60         try:
61             st = float(sg["start_time"]) + clip_start

```

28. user (2026-06-18T01:28:51.166Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """
21        a_start, a_end = a
22        b_start, b_end = b
23        if a_end <= a_start or b_end <= b_start:
24            return 0.0
25        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26        if inter <= 0.0:
27            return 0.0

```

```

28         union = (a_end - a_start) + (b_end - b_start) - inter
29         return inter / union if union > 0 else 0.0
30
31
32     @dataclass
33     class Match:
34         """One matched predicted?ground-truth pair."""
35         pred_index: int
36         gt_index: int
37         iou: float
38
39
40     @dataclass
41     class MatchResult:
42         """Outcome of matching predictions to ground truth at a given IoU threshold."""
43         matches: List[Match] = field(default_factory=list)
44         unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45         unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
46 (misses)
47
48     def match_intervals(preds: List[Interval], gts: List[Interval],
49                       iou_threshold: float = 0.5) -> MatchResult:
50         """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52         Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53         are claimed highest-IoU first, and each prediction/GT can be used once. This
54         keeps a single prediction from being credited for two ground-truth rallies
55         (and vice versa).
56         """
57         candidates = []
58         for pi, p in enumerate(preds):
59             for gi, g in enumerate(gts):
60                 iou = interval_iou(p, g)
61                 # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
62                 # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63                 if iou > 0.0 and iou >= iou_threshold:
64                     candidates.append((iou, pi, gi))
65         # Highest IoU first; ties broken by indices for determinism.
66         candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68         used_pred, used_gt = set(), set()
69         matches: List[Match] = []
70         for iou, pi, gi in candidates:
71             if pi in used_pred or gi in used_gt:
72                 continue
73             used_pred.add(pi)
74             used_gt.add(gi)
75             matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77         unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78         unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79         return MatchResult(matches=matches,
80                           unmatched_pred_indices=unmatched_pred,
81                           unmatched_gt_indices=unmatched_gt)
82
83
84     @dataclass
85     class Score:
86         """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87         iou_threshold: float
88         num_pred: int
89         num_gt: int
90         true_positives: int

```

```

91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float          # over matched pairs (0.0 if none)
97     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104     def score(preds: List[Interval], gts: List[Interval],
105              iou_threshold: float = 0.5,
106              match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114         tp = len(mr.matches)
115         fp = len(mr.unmatched_pred_indices)
116         fn = len(mr.unmatched_gt_indices)
117
118         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
119         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
120         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
121     else 0.0
122
123         if mr.matches:
124             mean_iou = sum(m.iou for m in mr.matches) / tp
125             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
126 mr.matches) / tp
127             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
128 mr.matches) / tp
129         else:
130             mean_iou = mean_start_err = mean_end_err = 0.0
131
132         return Score(
133             iou_threshold=iou_threshold,
134             num_pred=len(preds), num_gt=len(gts),
135             true_positives=tp, false_positives=fp, false_negatives=fn,
136             precision=precision, recall=recall, f1=f1,
137             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
138         )
139
140     def pr_curve(preds: List[Interval], gts: List[Interval],
141                 thresholds: Optional[List[float]] = None) -> List[Score]:
142         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
143
144         Lets you see strict-vs-lenient behaviour rather than a single operating point.
145         """
146         if thresholds is None:
147             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
148         return [score(preds, gts, iou_threshold=t) for t in thresholds]

```

29. assistant (2026-06-18T01:29:03.334Z)

Now let me carefully examine a potential bug I noticed in `serve\_contrast.py`. Look at

```
`gate_a_scores_at`:
```

```
```python
windows = W.build_windows(points, fps, frame_width, preset)
intervals = W.windows_to_intervals(windows)
...
est = rally_gate.estimate_play_axis(points)
gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0, estimate=est)
crossings = [g.crossings for g in gated]
...
kept = [iv for iv, c in zip(intervals, crossings) if c >= min_crossings]
```
```

This looks consistent ? `gate\_windows` produces one `GatedWindow` per interval in order, so `crossings` aligns 1:1 with `intervals`. That's fine.

Now let me re-examine the R5 blob-fallback / R4 trim interaction for a real ordering bug. In `\_blob\_fallback\_chop`, the groups passed in could have been reordered by R4? No ? R4 is a per-group map preserving order. But `\_split\_overlong\_groups` does `out.sort(key=lambda grp: grp[0][0])`.

Let me look harder at the `forced\_flags` alignment after the min_window_duration filter. The final filter at line 388 drops windows below min_window. But `forced\_flags` is consumed in the zip BEFORE filtering, so the `forced\_cut` flag is correctly attached per-window. That's fine.

Let me now examine the `\_finalize` group's `mean\_velocity` when a group could be empty. Let me look at whether `\_blob\_fallback\_chop` or `\_split\_overlong\_groups` can produce an empty group. In `\_split\_overlong\_groups` hard_cap path: `split\_i = min(range(1, len(g)), ...)`. Since `len(g) >= 2` is guaranteed (the `len(g) < 2` check appends and continues), `range(1, len(g))` is non-empty, `split\_i >= 1`, so `g[:split\_i]` is non-empty and `g[split\_i:]` is non-empty. Good.

Let me look at `\_derive\_video\_path` and the `fusion\_golden` `net\_norm` calc for a div-by-zero or sign issue. Let me also check `calibrate\_wasb.load\_golden\_gts` and `load\_gt\_intervals` to verify the contract in `freeze\_real\_fixture`.

30. user (2026-06-18T01:29:04.057Z)

```
1      """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3      The repo historically carried two incompatible loaders:
4      - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5        ``curate export`` format), raises on bad rows;
6      - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7        (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9      A file in one convention fed to the other loader yielded zero/garbage intervals with no
10     error. This module accepts BOTH conventions (header-mapped, else positional), so every
11     quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12     flows through one reader. Both legacy entry points now delegate here.
13
14     Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15     - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16       columns ? rally_number, ending_reason, sport, ? ? are ignored);
17     - no header: columns 0,1 positionally.
18     Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19     (``annotations.parse_timestamp``).
20     """
21     import csv
22     import logging
23     from typing import List, Optional, Tuple
24
25     from backend.eval.annotations import parse_timestamp
26
27     logger = logging.getLogger(__name__)
```

```

27
28     Interval = Tuple[float, float]
29
30     _START_NAMES = ("start", "start_time")
31     _END_NAMES = ("end", "end_time")
32
33
34     def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35         """If `row` is a header naming start/end columns, return their indices, else
None."""
36         cells = [(c or "").strip().lower() for c in row]
37         start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38         end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39         if start_idx is not None and end_idx is not None:
40             return start_idx, end_idx
41         return None
42
43
44     def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45         """Load GT rally intervals from either golden-CSV convention.
46
47         strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48         (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49         strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50         skipped ? a silently shrinking GT set flatters precision.
51
52         Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53         (they break one-to-one matching and usually indicate a labeling error).
54         """
55         intervals: List[Interval] = []
56         skipped = 0
57         cols: Optional[Tuple[int, int]] = None    # (start_idx, end_idx); None until detected
58         saw_header = False
59
60         with open(csv_path, newline="", encoding="utf-8-sig") as f:
61             for lineno, row in enumerate(csv.reader(f), start=1):
62                 if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                     continue
64                 if cols is None and not saw_header:
65                     header = _header_columns(row)
66                     if header is not None:
67                         cols = header
68                         saw_header = True
69                         continue    # consume the header row
70                 cols = (0, 1)    # headerless: positional start,end
71                 assert cols is not None    # every branch above assigns it
72                 si, ei = cols
73                 try:
74                     if len(row) <= max(si, ei):
75                         raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                     start = parse_timestamp(row[si])
77                     end = parse_timestamp(row[ei])
78                     if start < 0:
79                         raise ValueError(f"negative start ({start})")
80                     if start >= end:
81                         raise ValueError(f"start ({start}) >= end ({end})")
82                 except ValueError as e:
83                     if strict:
84                         raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                     skipped += 1
86                     continue
87                 intervals.append((start, end))
88
89         if skipped:

```



```

90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

31. assistant (2026-06-18T01:29:13.992Z)

Now I want to focus on a subtle potential bug in ``freeze_real_fixture``'s negative-label guard interacting with ``gt_hash``. The provenance is built with ``gt_hash(reset_gts)`` (line 737) using the in-memory reset labels. But then ``write_labels_csv(reset_gts, ...)`` writes them at 6dp, and ``replay_fixture`` reads them back via ``load_gt_intervals`` and computes ``gt_hash(gts)`` on the re-read labels.

If the in-memory ``reset_gts`` and the re-read (6dp serialized) labels differ, the provenance ``gt_hash`` and the expected.json ``gt_hash`` would differ. But that's a separate field; provenance `gt_hash` isn't compared against expected. Let me not chase that.

Let me check ``gt_hash`` and ``regression.py`` to understand how `gt_hash` rounds, since the negative-label guard checks ``s < 0`` on the in-memory `reset_gts` but the *written* labels are 6dp-rounded. Actually ``_reset_labels`` already applies ``_round`` so `reset_gts` is already 6dp. Fine.

Let me look more carefully at the R4 ``_trim_inactive_tail`` two-pointer for an off-by-one. The window condition is ``frames[lo] <= frames[i] - trail_frames``. This is a half-open trailing window. Let me trace the test case and check correctness against the docstring claim "trailing ``trail_frames`` window".

Let me now examine ``count_net_crossings`` and ``_points_in`` for the deadband/frame boundary, and look at whether ``estimate_play_axis`` handles the all-invisible case.

32. user (2026-06-18T01:29:14.868Z)

```

37: def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38-     """Stable short hash of the GT intervals ? detects a baseline taken against different
labels."""
39-     norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40-     return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41-
42-
43- @dataclass
44- class RegressionResult:
45-     video_id: str
46-     stage: str
47-     iou_threshold: float
48-     precision: float
49-     recall: float
50-     f1: float
51-     # Baseline values compared against (None when no baseline existed yet).
52-     baseline_precision: Optional[float]
53-     baseline_recall: Optional[float]
54-     tolerance: float
55-     regressed: bool
56-     reasons: List[str]
57-     # True only when a baseline existed to compare against. Distinguishes a genuine PASS

```

33. assistant (2026-06-18T01:29:24.711Z)

Now let me carefully look at the ``serve_contrast.py`` ``gate_a_scores_at`` one more time. There's a

subtle inconsistency: `build\_windows` (via the preset) applies the cap/R4/R5 transforms, but `gate\_windows` is called with `min\_crossings=0` to GET crossings, then `kept` is filtered by `min\_crossings`. The `intervals` come from `windows\_to\_intervals(windows)`. This is consistent.

However ? note that the preset passed is `W.PROPOSAL` and `W.SERVED`. `build\_windows` passes the preset's R4/R5/cap knobs. For PROPOSAL these are all OFF/default. For SERVED they're read from config defaults. Fine.

Let me now look at the `distill\_local.build\_candidates` interaction with the golden_fixtures replay. The replay zips `wins` and `gated` in `build\_candidates`. `gate\_windows` returns one GatedWindow per interval, so `zip(wins, gated)` is 1:1. Good.

Let me look at a more promising candidate: in `golden\_fixtures.replay\_fixture`, the candidates loop builds `shuttle` dict, and `net\_crossings` is the integer count from `feature\_rows[j]`. But `build\_candidates` returns `X` where `net\_crossings` is `float(g.crossings)`. Now let me check ? in `freeze\_real\_fixture`, the trajectory is reset to start near frame 0, but **the labels** are reset by `t0\_sec` (seconds). The trajectory frame reset uses integer `t0\_frame`. Note the comment about the ONE ULP. This is documented as an honest limit, not a bug.

Let me look at one thing in `fusion\_golden.extract\_video` that could be a real bug ? the `net\_norm` computation and the `shuttle\_in` filter using `win\_frames`:

34. user (2026-06-18T01:29:26.155Z)

```
1      """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3      Wires the already-tested pieces together:
4      WASB trajectory CSV (Frame,Visibility,X,Y)
5      -> TrackNetRunner.trajectory_to_action_windows(cfg) [predict_fn]
6      -> backend.eval.calibration(improvement_report / cross_validate)
7
8      Run (after a full-video WASB pass exists):
9      python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10         --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an optimistic upper bound. The trustworthy number is the
14      cross-validated held-out recall (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import argparse
20      from typing import List, Tuple, Callable
21
22      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
23      from backend.eval.calibration import (
24          select_best, improvement_report, cross_validate, evaluate,
25      )
26
27      Interval = Tuple[float, float]
28      Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
29
30      BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33      def load_golden_gts(csv_path: str) -> List[Interval]:
34          """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36          Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37          start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38          driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
```

```

39     """
40     from backend.eval.gt_loader import load_gt_intervals
41     return load_gt_intervals(csv_path, strict=False)
42
43
44     def make_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
47
48         def predict_fn(cfg: Config) -> List[Interval]:
49             vt, mg, mwd = cfg
50             wins = TrackNetRunner.trajectory_to_action_windows(
51                 points, fps=fps, frame_width=frame_width,
52                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53             )
54             return [(w["start"], w["end"]) for w in wins]
55
56         return predict_fn
57
58
59     def default_grid() -> List[Config]:
60         return [(vt, mg, mwd)
61                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62                 for mg in (1.0, 2.0, 3.0)
63                 for mwd in (1.0, 2.0, 3.0)]
64
65
66     def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
67            iou: float = 0.5) -> dict:
68         gts = load_golden_gts(labels_csv)
69         if not gts:
70             raise SystemExit(f"no usable golden rallies in {labels_csv}")

```

35. assistant (2026-06-18T01:29:44.430Z)

Now let me dig into a precise area: the R4 `\_trim\_inactive\_tail` two-pointer window semantics versus the docstring, and whether the trailing-window density check has an off-by-one. Let me trace the algorithm carefully.

The docstring says: "The rally is 'on' at frame `i` while its trailing `trail\_frames` window holds $\geq$ `min\_density` fraction of fast frames". The window for frame `i` is frames `(frames[i] - trail\_frames, frames[i]]`. The condition `while frames[lo] <= frames[i] - trail\_frames: lo += 1` removes frames at or before `frames[i] - trail\_frames`. So after the loop, `frames[lo] > frames[i] - trail\_frames`, meaning the window is `(frames[i] - trail\_frames, frames[i]]` ? half-open at the lower bound. `win\_count = i - lo + 1`. This is consistent and correct.

Now let me think about a real bug. Consider `\_trim\_inactive\_tail` when there are **frame gaps** within a group (frames not consecutive). The group is built from `active` frames which can have gaps (active frames within merge_gap). The two-pointer uses `frames[lo] <= frames[i] - trail\_frames`, accounting for actual frame numbers. That's correct.

Let me now look very carefully at the `\_split\_overlong\_groups` hard_cap midpoint chop for a potential **infinite loop**. When `hard\_cap=True` and a group has no qualifying gap, it splits at the midpoint frame. Consider a degenerate group of exactly 2 frames where the span $>$ max_win_frames. `len(g) >= 2`, span $>$ max_win_frames. `best\_gap` = the single gap. If `best\_gap >= min\_split\_frames`, it splits there normally. If `best\_gap < min\_split\_frames` (the gap itself is large but below the floor ? impossible since span $>$ max_win_frames... wait, with 2 frames, span == the single gap).

So if span $>$ max_win_frames but the single gap $<$ min_split_frames ? that requires max_win_frames $<$ min_split_frames. Then hard_cap: `mid = (g[0][0]+g[-1][0])/2`, `split\_i = min(range(1,2), ...) = 1`. So `g[:1]` and `g[1:]` ? each a single-frame group. Single frame groups have `len < 2`, so they go to `out`. Terminates. Good.

Let me look at a 3+ frame group where all frames are at the SAME position... no, frames are monotonic increasing (sorted unique by construction from active list). Let me verify active frames are unique. In the active-building loop, each `p.frame` is appended at most once. Points are sorted by frame; could two points have the same frame? `parse\_trajectory\_csv` doesn't dedupe. If two rows share a Frame... let me check. Actually, let me move on ? that's an unlikely edge.

Let me look at the genuinely suspicious area: `**`min_window_duration` filter applied AFTER R5 forced cuts**`. The R5 blob chop produces pieces each ? cap. But pieces could be SHORTER than ``min_window_duration`` and get dropped, while still being flagged ``forced_cut``. More importantly ? consider the test ``test_window_blob_fallback_rescues_blob_through_api`` asserts ``all(w.get("forced_cut") for w in on)``. The filter drops sub-min windows. If a forced piece is < min_window, it's dropped ? fine, not a bug.

Let me examine the actual interaction I'm now most suspicious of ? `**`forced_flags` misalignment when `min_window_duration` and R4 both modify groups**`. Actually ``forced_flags`` is built in R5 and zipped right after. Let me re-read lines 377-388 precisely.

36. user (2026-06-18T01:29:46.061Z)

```
358         if max_window_duration > 0:
359             groups = TrackNetRunner._split_overlong_groups(
360                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
361                 hard_cap=window_hard_cap)
362
363         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368         if window_inactivity_end > 0:
369             tw = int(window_inactivity_end * fps)
370             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                 inactivity_rally_thresh) for g
in groups]
372
373         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377         forced_flags = [False] * len(groups)
378         if window_blob_fallback and max_window_duration > 0:
379             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380                 groups, int(max_window_duration * fps), window_blob_factor)
381
382         windows = []
383         for g, forced in zip(groups, forced_flags):
384             w = _finalize(g)
385             if forced:
386                 w["forced_cut"] = True
387             windows.append(w)
388         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
```

```
float]]:
```

37. assistant (2026-06-18T01:30:01.473Z)

The R5 alignment is fine. Now let me check the `\_split\_overlong\_groups` argument order versus the parameter names ? there's a subtle ordering concern. Line 370-371:

```
```python
groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
 inactivity_rally_thresh) for g in groups]
...

_trim_inactive_tail(group, trail_frames, min_density, rally_thresh)`. So `tw`?trail_frames,
`inactivity_min_density`?min_density, `inactivity_rally_thresh`?rally_thresh. Correct order.
```

Now let me look at the `build\_windows` call in windowing.py ? it passes  
`inactivity\_rally\_thresh` and `inactivity\_min\_density` to `trajectory\_to\_action\_windows`. Let me  
verify the parameter names match. In `trajectory\_to\_action\_windows` signature:  
`inactivity\_min\_density: float = 0.34, inactivity\_rally\_thresh: float = 0.08`. In  
`build\_windows`:

```
```python
window_inactivity_end=preset.inactivity_end,
inactivity_rally_thresh=preset.inactivity_rally_thresh,
inactivity_min_density=preset.inactivity_min_density,
...

```

These are keyword args, so they match correctly.

Now let me examine the most suspicious thing I've seen ? let me revisit
`\_split\_overlong\_groups`. When `hard\_cap=True` and the group has a gap but `best_gap <
min_split_frames`, it falls into the hard_cap midpoint chop. But after midpoint chop, the two
halves each still might have that same sub-floor gap and exceed max_win. They get re-pushed to
`work`. Could the midpoint always land such that one half never shrinks? `split\_i` is the frame
closest to midpoint. With `len(g) >= 2`, `split\_i` in `[1, len(g)-1]`, so both halves have `len
>= 1`. If `len(g) == 2`, both halves are single-frame ? terminate. For `len(g) >= 3`, both
halves strictly smaller than original (since `1 <= split\_i <= len-1`). So each iteration
strictly reduces group size; terminates. Good.

Now I want to verify one real concern in `golden\_fixtures.\_fmt`/`\_round` with NaN. If a feature
is NaN, `round(nan)` returns nan, `nan == 0.0` is False. Not reachable with synthetic data.
Skip.

Let me check the `freeze\_real\_fixture` negative-label guard more carefully ? there may be a real
bug where `gt\_hash` in provenance uses `reset\_gts` (in-memory floats) but the labels written/re-
read may round differently, making provenance.gt_hash inconsistent with expected.gt_hash. But
this is not a crash and not compared. Let me instead verify the `\*\_resolve\_dir` behavior in
`replay\_fixture` when given a `tmp\_dir` Path. In `freeze\_real\_fixture`,
`replay\_fixture(tmp\_dir)` is called. `\_resolve\_dir` returns the Path as-is. Then `slug =
fdir.name` = `out_slug`. Good.

Let me now look at the broader test suite for golden fixtures and the real-fixtures test to see
if there's an untested path.

38. user (2026-06-18T01:30:02.704Z)

```
1      """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3      These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4      minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5      its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7      Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
```

```

8         We reuse M1's synthetic generators (`builtin_specs` / `make_synthetic_trajectory`
and the
9         ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10        prove:
11            * the refusal gate blocks every unsafe combination (M2);
12            * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13            * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14            ``control_segments`` shift toward 0);
15            * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17        $6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18        are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
19        """
20        import pytest
21
22        from backend.eval import golden_fixtures as gf
23
24
25        # -----
26        # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27        # -----
28
29
30        def _write_source(tmp_path, *, frame_offset: int = 0):
31            """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33            Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34            ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35            capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36            """
37            spec = gf.builtin_specs()[0] # clean_single_rally
38            fps, fw = spec.fps, spec.frame_width
39            rows = gf.make_synthetic_trajectory(spec)
40            if frame_offset:
41                rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
42            gts = list(spec.gts)
43            if frame_offset:
44                off_sec = frame_offset / fps
45                gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
46
47            traj = tmp_path / "src_trajectory.csv"
48            labels = tmp_path / "src_labels.csv"
49            gf.write_trajectory_csv(rows, traj)
50            gf.write_labels_csv(gts, labels)
51            return traj, labels, fps, fw, gts
52
53
54        # -----
55        # M2 ? refusal gate
56        # -----
57
58
59        @pytest.mark.parametrize(
60            "label, kwargs, why",
61            [
62                ("scenario", {"minors_free": False, "owned": True, "license": "owned"},
"minors_free=False"),
63                ("scenario", {"minors_free": True, "owned": False, "license": "owned"},
"owned=False"),

```

```

64         ("scenario", {"minors_free": True, "owned": True, "license": ""}, "license=''",
65         ("scenario", {"minors_free": True, "owned": True, "license": "  "},
"license=blank"),
66         ("scenario", {"minors_free": True, "owned": True, "license": None},
"license=None"),
67         # red-team #1: a free-text identifier label (name / event / spaces /
punctuation) is refused.
68         ("Lin Dan vs Lee, India Open 2024",
69         {"minors_free": True, "owned": True, "license": "owned"}, "label=player-name
free text"),
70         ("mens singles final",
71         {"minors_free": True, "owned": True, "license": "owned"}, "label=spaces"),
72         ("Final_2024", {"minors_free": True, "owned": True, "license": "owned"},
"label=uppercase"),
73         ("", {"minors_free": True, "owned": True, "license": "owned"}, "label=empty"),
74     ],
75 )
76 def test_refusal_gate_blocks_unsafe(tmp_path, label, kwargs, why):
77     """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
blank license
78     AND a STRICT-slug label (no free-text identifier ? red-team #1).
79
80     The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
fixture path
81     must be impossible to reach silently. A refused freeze leaves NO partial fixture dir
(red-team
82     #3b: write-to-temp-then-promote).
83     """
84     traj, labels, fps, fw, _ = _write_source(tmp_path)
85     with pytest.raises(ValueError) as ei:
86         gf.freeze_real_fixture(
87             label, traj, labels,
88             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
89     # RefusalError is a ValueError subclass; the message must name a reason.
90     assert isinstance(ei.value, gf.RefusalError), why
91     assert "REFUSED" in str(ei.value), why
92     # Nothing was written for a refused freeze (no fixture dir, partial or otherwise).
93     assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
"fixtures").iterdir()), (
94         f"{why}: a refused freeze must not write any fixture")
95
96
97     # -----
98     # P1 ? de-attribution
99     # -----
100
101
102 def test_real_fixture_is_deattributed(tmp_path):
103     """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
104     no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
105     fixtures = tmp_path / "fixtures"
106     traj, labels, fps, fw, _ = _write_source(tmp_path)
107     expected = gf.freeze_real_fixture(
108         "clean_rally", traj, labels,
109         license="owned", minors_free=True, owned=True,
110         fps=fps, frame_width=fw, fixtures_dir=fixtures)
111
112     slug = expected["slug"]
113     assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"
114
115     prov = gf._read_json(fixtures / slug / "provenance.json")
116     assert prov["kind"] == "cleared_real"
117     assert prov["minors_free"] is True and prov["owned"] is True

```

```

118     assert prov["license"] == "owned"
119     assert prov["slug"] == slug
120     assert prov["label"] == "clean_rally" # strict-slug label is committed verbatim
121     # No attribution keys / no source filename anywhere in the provenance. `source_note`
is now
122     # DROPPED entirely (red-team #1b): operator free-text never reaches the committed
bytes.
123     for banned in ("video_id", "name", "filename", "path", "match", "capture_date",
"source_note"):
124         assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
125         prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
126         for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
127             assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
128
129         # expected.json carries no biometric/identity/non-shuttle stream.
130         exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
131         for forbidden in gf.FORBIDDEN_KEYS:
132             assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
133
134         # The fixture is registered in the manifest with kind=cleared_real.
135         manifest = gf.load_manifest(fixtures)
136         rows = [e for e in manifest if e["slug"] == slug]
137         assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
138
139
140     def test_supplied_slug_is_honored(tmp_path):
141         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
142         fixtures = tmp_path / "fixtures"
143         traj, labels, fps, fw, _ = _write_source(tmp_path)
144         expected = gf.freeze_real_fixture(
145             "scenario", traj, labels, slug="fx_custom00",
146             license="owned", minors_free=True, owned=True,
147             fps=fps, frame_width=fw, fixtures_dir=fixtures)
148         assert expected["slug"] == "fx_custom00"
149         assert (fixtures / "fx_custom00" / "expected.json").is_file()
150
151
152     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
153         """Freezing a second real fixture must APPEND to the manifest, not wipe the
first."""
154         fixtures = tmp_path / "fixtures"
155         traj, labels, fps, fw, _ = _write_source(tmp_path)
156         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
157                                     license="owned", minors_free=True, owned=True,
158                                     fps=fps, frame_width=fw, fixtures_dir=fixtures)
159         e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
160                                     license="owned", minors_free=True, owned=True,
161                                     fps=fps, frame_width=fw, fixtures_dir=fixtures)
162         slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
163         assert {e1["slug"], e2["slug"]} <= slugs
164
165
166     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
167         """source_note is ACCEPTED-BUT-NOT-PERSISTED (red-team #1b), yet still scanned
defensively ? a
168         path/name in it trips the positive scan and refuses (and leaves no partial dir, red-
team #3b)."""
169         fixtures = tmp_path / "fixtures"
170         traj, labels, fps, fw, _ = _write_source(tmp_path)
171         with pytest.raises(gf.RefusalError):
172             gf.freeze_real_fixture(

```



```

173         "scenario", traj, labels, slug="fx_leak0001",
174         license="owned", minors_free=True, owned=True,
175         fps=fps, frame_width=fw, fixtures_dir=fixtures,
176         source_note=str(traj)) # leaking the source path ? must be refused
177     # The refused freeze left no fixture dir behind (write-to-temp-then-promote).
178     assert not (fixtures / "fx_leak0001").exists(), "refused freeze must not leave a
partial dir"
179
180
181     def test_source_note_safe_is_not_persisted(tmp_path):
182         """A SAFE source_note (no path/name) is accepted but NEVER written to the committed
provenance
183         (red-team #1b: operator free-text is an attribution back-channel, dropped from
committed bytes)."""
184         fixtures = tmp_path / "fixtures"
185         traj, labels, fps, fw, _ = _write_source(tmp_path)
186         expected = gf.freeze_real_fixture(
187             "scenario", traj, labels, slug="fx_note0001",
188             license="owned", minors_free=True, owned=True,
189             fps=fps, frame_width=fw, fixtures_dir=fixtures,
190             source_note="benign generic note")
191         prov = gf._read_json(fixtures / expected["slug"] / "provenance.json")
192         assert "source_note" not in prov, "source_note must NOT be persisted into
provenance"
193         prov_text = (fixtures / expected["slug"] /
"provenance.json").read_text(encoding="utf-8")
194         assert "benign generic note" not in prov_text, "source_note text leaked into
committed bytes"
195
196
197     def test_freeze_real_label_slug_committed(tmp_path):
198         """A valid strict-slug label is committed verbatim; a free-text label is refused
(red-team #1a)."""
199         fixtures = tmp_path / "fixtures"
200         traj, labels, fps, fw, _ = _write_source(tmp_path)
201         ok = gf.freeze_real_fixture(
202             "mens-singles_final-01", traj, labels, slug="fx_lab00001",
203             license="owned", minors_free=True, owned=True,
204             fps=fps, frame_width=fw, fixtures_dir=fixtures)
205         prov = gf._read_json(fixtures / ok["slug"] / "provenance.json")
206         assert prov["label"] == "mens-singles_final-01"
207         with pytest.raises(gf.RefusalError):
208             gf.freeze_real_fixture(
209                 "Lin Dan vs Lee 2024", traj, labels, slug="fx_lab00002",
210                 license="owned", minors_free=True, owned=True,
211                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
212
213
214     def test_freeze_real_negative_reset_label_refused(tmp_path):
215         """A label timeline that predates the earliest trajectory frame ? negative reset
start ? refused
216         BEFORE any write, leaving no partial fixture dir (red-team #3a)."""
217         fixtures = tmp_path / "fixtures"
218         spec = gf.builtin_specs()[0]
219         fps, fw = spec.fps, spec.frame_width
220         # Trajectory starts at frame 300 (t0 ? 10s) but a label starts at t=0 ? reset start
< 0.
221         rows = [(f + 300, v, x, y) for (f, v, x, y) in gf.make_synthetic_trajectory(spec)]
222         traj = tmp_path / "src_trajectory.csv"
223         labels = tmp_path / "src_labels.csv"
224         gf.write_trajectory_csv(rows, traj)
225         gf.write_labels_csv([(0.0, 4.0)], labels)
226         with pytest.raises(gf.RefusalError) as ei:
227             gf.freeze_real_fixture(
228                 "scenario", traj, labels, slug="fx_neg00001",

```

```

229         license="owned", minors_free=True, owned=True,
230         fps=fps, frame_width=fw, fixtures_dir=fixtures, time_origin_reset=True)
231     assert "predates the trajectory" in str(ei.value)
232     assert not (fixtures / "fx_neg00001").exists(), "refused freeze must not leave a
partial dir"
233
234
235     def test_scan_forbidden_catches_label_csv_source_stem(tmp_path):
236         """red-team #4: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``)
must still
237         be caught ? the source-path scan uses EXACT schema-literal membership, never
substring-skip."""
238         # A source-path token "label" is NOT an exact schema literal, so it must trip the
scan when it
239         # appears in the written text. We feed it via a source path whose stem is "label".
240         text = '{"slug": "fx_x", "note": "label appears here"}'
241         with pytest.raises(gf.RefusalError) as ei:
242             gf._scan_forbidden(text, where="x/expected.json",
243                               source_paths=["/some/dir/label.csv"])
244         assert "source-path token" in str(ei.value) and "label" in str(ei.value)
245
246
247     def test_scan_forbidden_allows_exact_schema_literal(tmp_path):
248         """A source path that EXACTLY equals a schema literal (e.g. ``trajectory.csv``) is
not a leak ?
249         those bytes are written regardless, so the scan must not false-positive on them."""
250         text = '{"slug": "fx_x", "paths": {"trajectory": "trajectory.csv", "labels":
"labels.csv"}}'
251         # Source basename == schema literal "trajectory.csv": must NOT raise.
252         gf._scan_forbidden(text, where="x/expected.json",
source_paths=["/some/dir/trajectory.csv"])
253
254
255     # -----
256     # P1 (d) ? time-origin reset is metric-invariant
257     # -----
258
259
260     def _raw_score_of_frozen(fdir):
261         """Re-score a frozen fixture from its committed CSVs through the SHIPPED stack
WITHOUT the
262         snapshot's final 6dp rounding ? so the test sees the genuine metric, not the
serialization grid.
263
264         (``replay_fixture`` rounds every float to 6dp for the committed snapshot; on a sub-
second
265         time-origin reset two boundary errors can land on adjacent 6dp grid points ? a 1e-6
SERIALIZED
266         gap that is the grid, not a metric divergence. The raw score below exposes the true
delta.)
267         """
268         from backend.eval import distill_local
269         from backend.eval.experiment import CONTROL, VideoCandidates, predict
270         from backend.eval.gt_loader import load_gt_intervals
271         from backend.eval.metrics import score
272         intervals, feature_rows = distill_local.build_candidates(
273             str(fdir / "trajectory.csv"), fps=30.0, frame_width=1280.0, proposal=(0.02, 2.0,
2.0))
274         feats = {n: [r[i] for r in feature_rows] for i, n in enumerate(gf.SHUTTLE_FEATURES)}
275         gts = load_gt_intervals(str(fdir / "labels.csv"), strict=True)
276         preds = predict(CONTROL, VideoCandidates("x", intervals, feats, gts))
277         return score(preds, gts, 0.5)
278
279
280     # Fractional-second offsets (7, 11, 13 frames @30fps) plus whole-second (9000 = 300.0s)

```

```

and a few
281     # more sub-second offsets ? every one must be metric-invariant within _FLOAT_TOL (red-
team #2).
282     @pytest.mark.parametrize("offset_frames", [1, 7, 11, 13, 17, 9000])
283     def test_time_origin_reset_is_metric_invariant(tmp_path, offset_frames):
284         """A source frozen WITH reset scores within ``_FLOAT_TOL`` of the SAME source frozen
WITHOUT
285         reset, for ANY fractional-second offset ? and its absolute control_segments shift
toward 0
286         (severed timeline). Before fix #2 the boundary error drifted to 1.000000000001e-06
(just past
287         _FLOAT_TOL) on a sub-second offset; rounding t0_sec + each label endpoint to 6dp
before the
288         shift keeps the genuine (raw) delta at the float-error floor (~3.3e-7)."""
289         traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
290
291         with_reset = gf.freeze_real_fixture(
292             "scenario", traj, labels, slug="fx_reset_on",
293             license="owned", minors_free=True, owned=True,
294             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
295             time_origin_reset=True)
296         without_reset = gf.freeze_real_fixture(
297             "scenario", traj, labels, slug="fx_reset_off",
298             license="owned", minors_free=True, owned=True,
299             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
300             time_origin_reset=False)
301
302         # (1) Structural fields (counts/TP/FP/FN) are EXACTLY equal regardless of offset.
303         for k in ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives"):
304             assert with_reset["score"][k] == without_reset["score"][k], f"{k} changed under
reset"
305
306         # (2) Every float score field is metric-invariant within _FLOAT_TOL on the RAW (un-
rounded)
307         #     score ? the genuine metric, not the 6dp serialization grid (see
_raw_score_of_frozen).
308         raw_on = _raw_score_of_frozen(tmp_path / "with" / "fx_reset_on")
309         raw_off = _raw_score_of_frozen(tmp_path / "without" / "fx_reset_off")
310         for f in ("precision", "recall", "f1", "mean_iou", "mean_start_error",
"mean_end_error"):
311             delta = abs(getattr(raw_on, f) - getattr(raw_off, f))
312             assert delta <= gf._FLOAT_TOL, (
313                 f"offset={offset_frames}: {f} not metric-invariant under time-origin reset:
"
314                 f"|?|={delta!r} > _FLOAT_TOL={gf._FLOAT_TOL!r}")
315
316         # (3) A whole-SECOND offset is additionally BIT-identical on the serialized snapshot
(the 6dp
317         #     grid lands cleanly), proving the reset itself introduces no real divergence.
318         if offset_frames % int(fps) == 0:
319             assert with_reset["score"] == without_reset["score"], (
320                 "whole-second offset must be bit-identical on the serialized snapshot")
321
322         # (4) Absolute timeline differs ? the reset pulls control_segments toward 0.
323         seg_on = with_reset["control_segments"]
324         seg_off = without_reset["control_segments"]
325         assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
326         assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
327         shift = seg_off[0][0] - seg_on[0][0]
328         assert abs(shift - offset_frames / fps) < 1e-3, (
329             f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
330         # Reset trajectory starts at frame 0.

```

```

331     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
332                       .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
333     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
334
335
336     # -----
337     # P1 (e/f) ? round-trip / correct-by-construction
338     # -----
339
340
341     def test_freeze_real_round_trips_check(tmp_path):
342         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
343         expected.json
344         returns [] ? idempotent / correct-by-construction (the same path M1's --check
345         uses)."""
346         fixtures = tmp_path / "fixtures"
347         traj, labels, fps, fw, _ = _write_source(tmp_path)
348         expected = gf.freeze_real_fixture(
349             "scenario", traj, labels, slug="fx_rt000001",
350             license="owned", minors_free=True, owned=True,
351             fps=fps, frame_width=fw, fixtures_dir=fixtures)
352
353         fdir = fixtures / expected["slug"]
354         committed = gf._read_json(fdir / "expected.json")
355         recomputed = gf.replay_fixture(fdir)
356         assert gf.compare_expected(recomputed, committed) == [], (
357             "a just-frozen real fixture must replay-compare clean (correct-by-
358             construction)")
359
360
361     def test_freeze_real_uses_strict_label_loader(tmp_path):
362         """The labels are read through the STRICT gt_loader ? a malformed label row fails
363         loudly,
364         never silently shrinks the GT set."""
365         fixtures = tmp_path / "fixtures"
366         traj, labels, fps, fw, _ = _write_source(tmp_path)
367         # Corrupt one label row (end <= start) ? strict loader must raise.
368         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
369         with pytest.raises(ValueError):
370             gf.freeze_real_fixture(
371                 "scenario", traj, labels, slug="fx_strict01",
372                 license="owned", minors_free=True, owned=True,
373                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
374
375
376     def test_freeze_real_refuses_to_clobber_existing_dir(tmp_path):
377         """Freezing into a slug dir that already exists is refused (no silent overwrite)."""
378         fixtures = tmp_path / "fixtures"
379         traj, labels, fps, fw, _ = _write_source(tmp_path)
380         gf.freeze_real_fixture("a", traj, labels, slug="fx_dup000001",
381                                license="owned", minors_free=True, owned=True,
382                                fps=fps, frame_width=fw, fixtures_dir=fixtures)
383         with pytest.raises(ValueError):
384             gf.freeze_real_fixture("b", traj, labels, slug="fx_dup000001",
385                                    license="owned", minors_free=True, owned=True,
386                                    fps=fps, frame_width=fw, fixtures_dir=fixtures)
387
388
389     # -----
390     # FAILURE semantics ? the gates must FAIL loudly, not silently pass (red-team #10, #5,
391     # #7).
392     # -----
393
394     def _good_snapshot(tmp_path):

```

```

391         """Freeze a clean real fixture and return its committed expected.json snapshot
dict."""
392         fixtures = tmp_path / "fixtures"
393         traj, labels, fps, fw, _ = _write_source(tmp_path)
394         return gf.freeze_real_fixture(
395             "scenario", traj, labels, slug="fx_snap0001",
396             license="owned", minors_free=True, owned=True,
397             fps=fps, frame_width=fw, fixtures_dir=fixtures)
398
399
400     def test_compare_expected_detects_changed_float(tmp_path):
401         """A float field changed by > 1e-6 is a mismatch (characterization catches behaviour
drift)."""
402         import copy
403         snap = _good_snapshot(tmp_path)
404         other = copy.deepcopy(snap)
405         other["score"]["f1"] = snap["score"]["f1"] - 1e-3
406         assert gf.compare_expected(other, snap), "a >1e-6 float change must be a non-empty
mismatch list"
407
408
409     def test_compare_expected_detects_changed_int(tmp_path):
410         """A changed int/structural field is a mismatch."""
411         import copy
412         snap = _good_snapshot(tmp_path)
413         other = copy.deepcopy(snap)
414         other["score"]["true_positives"] = snap["score"]["true_positives"] + 1
415         assert gf.compare_expected(other, snap)
416
417
418     def test_compare_expected_detects_candidate_reorder(tmp_path):
419         """Reordering candidates is a mismatch (order is structural ? first-vs-second window
matters)."""
420         import copy
421         # Use a 2-candidate scenario so a reorder is observable.
422         spec = gf.builtin_specs()[1] # multi_rally_with_deadtime
423         rows = gf.make_synthetic_trajectory(spec)
424         traj = tmp_path / "src_trajectory.csv"
425         labels = tmp_path / "src_labels.csv"
426         gf.write_trajectory_csv(rows, traj)
427         gf.write_labels_csv(spec.gts, labels)
428         snap = gf.freeze_real_fixture(
429             "scenario", traj, labels, slug="fx_reord001",
430             license="owned", minors_free=True, owned=True,
431             fps=spec.fps, frame_width=spec.frame_width, fixtures_dir=tmp_path / "fixtures")
432         assert len(snap["candidates"]) >= 2
433         other = copy.deepcopy(snap)
434         other["candidates"] = list(reversed(other["candidates"]))
435         assert gf.compare_expected(other, snap)
436
437
438     def test_compare_expected_detects_extra_top_level_key(tmp_path):
439         """red-team #5: an EXTRA top-level key on one side is a mismatch (key-set
equality)."""
440         import copy
441         snap = _good_snapshot(tmp_path)
442         other = copy.deepcopy(snap)
443         other["unexpected_field"] = 1
444         mism = gf.compare_expected(other, snap)
445         assert mism and any("top-level" in m for m in mism), mism
446
447
448     def test_compare_expected_detects_extra_score_key(tmp_path):
449         """red-team #5: an EXTRA key inside the score block is a mismatch (key-set
equality)."""

```

```

450     import copy
451     snap = _good_snapshot(tmp_path)
452     other = copy.deepcopy(snap)
453     other["score"]["bogus_metric"] = 0.0
454     mism = gf.compare_expected(other, snap)
455     assert mism and any(m.startswith("score:") for m in mism), mism
456
457
458     def test_scan_forbidden_raises_on_forbidden_key():
459         """red-team #10: the FORBIDDEN_KEYS branch raises (a person/audio/pose stream in the
460         text)."""
461         with pytest.raises(gf.RefusalError) as ei:
462             gf._scan_forbidden('{"person": [1,2,3]}', where="x", source_paths=[])
463         assert "forbidden key" in str(ei.value)
464
465     def test_scan_forbidden_raises_on_md5():
466         """red-team #10: the MD5 branch raises (a 32-hex video_id shape is an attribution
467         back-channel)."""
468         md5 = "0123456789abcdef0123456789abcdef"
469         with pytest.raises(gf.RefusalError) as ei:
470             gf._scan_forbidden('{"slug": "%s"}' % md5, where="x", source_paths=[])
471         assert "MD5" in str(ei.value)
472
473     def test_refresh_snapshot_no_target_raises(tmp_path):
474         """red-team #10: refresh_snapshot with neither slug nor all raises ValueError."""
475         with pytest.raises(ValueError):
476             gf.refresh_snapshot(fixtures_dir=tmp_path)
477
478
479     def test_refresh_snapshot_cli_requires_reason(capsys):
480         """red-team #6: --refresh-snapshot --all WITHOUT --reason is a CLI error (argparse
481         exits 2)."""
482         with pytest.raises(SystemExit) as ei:
483             gf.main(["--refresh-snapshot", "--all"])
484         assert ei.value.code == 2 # argparse ap.error exit code
485         err = capsys.readouterr().err
486         assert "--reason" in err
487
488     def test_load_fixture_guards_provenance_schema_independently(tmp_path):
489         """red-team #7: a provenance schema_version > CURRENT (with expected sv in-range)
490         still raises ?
491         the two blocks are guarded INDEPENDENTLY, not masked by each other."""
492         fixtures = tmp_path / "fixtures"
493         traj, labels, fps, fw, _ = _write_source(tmp_path)
494         expected = gf.freeze_real_fixture(
495             "scenario", traj, labels, slug="fx_sv000001",
496             license="owned", minors_free=True, owned=True,
497             fps=fps, frame_width=fw, fixtures_dir=fixtures)
498         slug = expected["slug"]
499         # expected.json stays at the current schema; bump ONLY provenance to a future
500         version.
501         prov = gf._read_json(fixtures / slug / "provenance.json")
502         prov["schema_version"] = 99
503         gf._write_json(fixtures / slug / "provenance.json", prov)
504         with pytest.raises(ValueError) as ei:
505             gf.load_fixture(slug, fixtures_dir=fixtures)
506         assert "provenance schema_version 99" in str(ei.value)
507

```

39. assistant (2026-06-18T01:30:14.167Z)

This is a very thoroughly tested module. The golden_fixtures path is well-guarded. Let me now

focus more sharply on areas with weaker test coverage. Let me check the `serve\_contrast.py`
`merge\_gap` default discrepancy and the `gate\_a\_scores\_at` ? there's a potential bug where
`crossings` are computed on `intervals` BEFORE the cap/R4/R5 transforms but... no, `intervals`
come from `windows` which already had transforms applied. Let me verify there's no issue with
`gate\_windows` order versus `merge\_overlaps`.

Actually, let me look at one real concern in `serve\_contrast.gate\_a\_scores\_at`: the `gated` is
computed via `gate\_windows(points, intervals, ...)` . `intervals` is
`windows\_to\_intervals(windows)` . But after building windows under SERVED preset (which may apply
R4/R5/cap), the resulting `intervals` are post-transform. Then `crossings[i]` corresponds to
`intervals[i]` . The `kept` filter zips `intervals` with `crossings` . Consistent.

Let me now look at whether there's an issue with `count\_net\_crossings` deadband when `span ==
0` . In `estimate_play_axis`, `_spread` returns 0.0 for `len < 2` . Then `deadband = 0.06 * 0 =
0` . Then in `count_net_crossings`, `abs(v) < 0` is never true, so all points count. Not a crash.

Let me look at the `\_finalize` function division ? `mean\_velocity = sum(vels)/len(vels)` . Could
`len(vels)` be 0? `group` always has ?1 element (group starts with `[active[0]]` or pieces with
?1). So `len >= 1` . Fine.

Let me examine `fusion\_golden.extract\_video` more carefully ? the `win\_frames` uses
`round(s\*fps)` and the shuttle filter `sf <= p.frame <= ef` . There's a potential concern:
`detector.detections\_over\_windows(video\_path, win\_frames, ...)` returns `per\_window` ? is it
guaranteed `len(per\_window) == len(intervals)` ? Let me check the PersonDetector contract.

40. user (2026-06-18T01:30:14.930Z)

```
backend\pipeline\detectors\person_detector.py:305: def detections_over_windows(self,
video_path: str, windows: List[Tuple[int, int]],
backend\pipeline\detectors\person_detector.py-306-                                     frame_skip:
int = 3,
backend\pipeline\detectors\person_detector.py-307-                                     progress_cb:
Optional[Callable[[int, int], None]] = None
backend\pipeline\detectors\person_detector.py-308-                                     ) ->
List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-309-         """Per-window person
detections via ONE sequential pass over the video.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-310-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-311-         Opens the video ONCE and reads
forward ? ``grab()`` to advance, ``retrieve()`` +
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-312-         detect only on sampled frames
inside a window ? instead of ``detections_per_frame``'s
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-313-         per-window open +
``POS_FRAMES`` seek, which on a long video degrades to
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-314-         decode-from-start per window
(0(window_index) ? the fusion-P3 extraction bottleneck;
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-315-         172 windows took ~80 min).
``windows`` = ``[(start_frame, end_frame), ...]``; returns a
backend\pipeline\detectors\person_detector.py-316-         list aligned to it, each ``{"frames":
{idx: [(tid,bbox)]}, "fps", "frame_width",
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-317-         "frame_height"}``` (same shape
as ``detections_per_frame``). Each window gets its own
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-318-         window-local ByteTrack.
``progress_cb(frames_done, total_frames)`` (called every ~256
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-319-         frames) drives progress
```

```

logging.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-320-      """
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-321-      self.lazy_load_model()
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-322-      cap =
cv2.VideoCapture(video_path)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-323-      n = len(windows)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-324-      fps =
cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
backend\pipeline\detectors\person_detector.py-325-      if fps <= 0:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-326-          fps = 30.0
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-327-      fw =
cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-328-      fh =
cap.get(cv2.CAP_PROP_FRAME_HEIGHT) if cap.isOpened() else 0.0
backend\pipeline\detectors\person_detector.py-329-      base = {"fps": fps, "frame_width":
float(fw), "frame_height": float(fh)}
backend\pipeline\detectors\person_detector.py-330-      if not cap.isOpened() or n == 0:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-331-          cap.release()
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-332-      return [dict(frames={},
**base) for _ in range(n)]
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-333-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-334-      step = max(frame_skip, 1)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py-335-      score_thresh =
self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
--
tests\test_fusion_compare.py:88:      def detections_over_windows(self, video, windows,
frame_skip, progress_cb=None):
tests\test_fusion_compare.py-89-          return [{"frames": {sf: [(1, (10.0, 10.0, 30.0,
40.0))]},
tests\test_fusion_compare.py-90-                  "fps": 30.0, "frame_width": 1920.0,
"frame_height": 1080.0}
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-91-
for (sf, ef) in windows]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-92-
tests\test_fusion_compare.py-93-      spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0,
"frame_width": 1920.0,
tests\test_fusion_compare.py-94-            "labels": "foo.rallies.csv"}
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-95-      data =
fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3, iou=0.5)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-96-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-97-      assert
data["name"] == "t"
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-98-      assert
data["gts"] == [[0.0, 10.0]]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-99-      assert
len(data["candidates"]) == 2
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-100-      c0 =
data["candidates"][0]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-101-      assert
set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-102-      assert
set(c0["person"]) == set(PERSON_FEATURE_NAMES)

```



```

C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-103-   assert
c0["label"] == 1                               # (0,10) matches the gt -> positive
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-104-   assert
data["candidates"][1]["label"] == 0           # (40,41) -> negative
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-105-   assert
all(isinstance(v, float) for v in c0["person"].values())
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-106-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-107-
tests\test_fusion_compare.py-108-def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-109-
"""run() skips a video whose JSON already exists (resume a long run); --fresh redoes all."""
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-110-   from
backend.eval import fusion_golden
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-111-   out_dir
= str(tmp_path / "out")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-112-   os.makedirs(out_dir)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-113-   manifest = [
tests\test_fusion_compare.py-114-       {"name": "alpha", "trajectory": "a.csv", "fps": 30.0,
"frame_width": 1920.0, "labels": "a.rallies.csv"},
tests\test_fusion_compare.py-115-       {"name": "beta", "trajectory": "b.csv", "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-116-   ]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_compare.py-117-   manifest_path = str(tmp_path / "m.json")
tests\test_fusion_compare.py-118-   with open(manifest_path, "w", encoding="utf-8") as f:

```

41. assistant (2026-06-18T01:30:31.987Z)

The `detections\_over\_windows` returns a list aligned 1:1 with windows. Good.

Now let me look at the actual distill_local `build\_candidates` interaction. The replay in golden_fixtures calls `build\_candidates` which internally re-parses the trajectory CSV. But in fusion_golden, `extract\_video` calls `parse\_trajectory\_csv(traj)` AND `build\_candidates(traj, ...)` which ALSO parses. That's a double parse but not a bug.

Let me now zoom into a precise concern in `distill\_local.build\_candidates`: line 61-72. `gate\_windows(points, intervals, fps, min\_crossings=0)`. The `intervals` come from `wins`. `zip(wins, gated)` is 1:1. But wait ? `gate\_windows` calls `\_points\_in(points, fps, s, e)` which filters `start\*fps <= p.frame <= end\*fps`. The intervals' start/end are in seconds derived from `frame/fps + chunk\_start`. With `chunk\_start=0` (default), `start = group[0][0]/fps`, so `start\*fps = group[0][0]` (modulo float). Fine.

I've done a thorough trace. Let me now check the one area I haven't fully verified ? the `\_split\_overlong\_groups` non-hard-cap path with `out.sort` versus the R5 `\_blob\_fallback\_chop` which does NOT do a final global sort. Let me check: after `\_split\_overlong\_groups` returns sorted groups, R4 maps preserving order, then `\_blob\_fallback\_chop` iterates groups in order and `out.extend(pieces)` where pieces are sorted within. So global order is preserved. But there's a subtle issue: `\_blob\_fallback\_chop` does NOT re-sort globally. If the input groups were already sorted (they are, post `\_split\_overlong\_groups`), and each group's pieces are sorted and inserted in place, the global order is maintained. Good.

But wait ? what if `max\_window\_duration == 0` (cap OFF) but R4 is ON? Then `\_split\_overlong\_groups` is NOT called (line 358 guard), so `groups` is NOT sorted by that path. The original `groups` from line 348-356 are built in active-frame order, which IS sorted by frame. So still sorted. Fine.

Let me look at one final candidate ? the `e1\_probe.cluster\_hits` for an off-by-one in the final cluster. Let me trace: after the loop, if `cnt >= min\_hits`, append `(start-pad, last+pad)`. The loop updates `last` and `cnt` correctly. Looks correct.

Let me check `\_hits\_in` and `match\_intervals(wasb, gts, iou)` in e1_probe ? `mr.unmatched\_gt\_indices` gives missed GTs. `missed = [gts[i] for i in mr.unmatched\_gt\_indices]`. Correct.

Let me now carefully re-examine the `serve\_contrast` `merge\_gap` default of `1.0` vs the docstring. Actually I want to verify the most subtle thing: in `gate\_a\_scores\_at`, the `est` is computed via `estimate\_play\_axis(points)` returning a 3-tuple, passed as `estimate=est` to `gate\_windows`. `gate\_windows` unpacks `axis, net, span = estimate`. Correct 3-tuple.

Let me verify there are no real bugs by checking `promotion.promote\_if\_served\_safe` signature against the `serve\_contrast` call.

42. user (2026-06-18T01:30:32.830Z)

```
110: def promote_if_served_safe(
111-     served_a: Sequence[float],
112-     served_b: Sequence[float],
113-     *,
114-     proposal_a: Optional[Sequence[float]] = None,
115-     proposal_b: Optional[Sequence[float]] = None,
116-     structural: bool = False,
117-     min_n: int = 5,
118-     p_threshold: float = 0.05,
119-     served_regress_eps: float = 0.0,
120-     n_boot: int = 2000,
121-     seed: int = 0,
122-) -> ServedGateResult:
123-     """Promote a cue ONLY if it does not regress at the SERVED windowing (the foundational
124-     rule).
125-     The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign
126-     test) is
127-     applied to the **served** scores ? the operating point production runs ? so a proposal-
128-     only
129-     win can never carry a promotion. The optional `proposal_` scores are scored too and
130-     reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
131-     their
132-     own).
133-     Args:
134-         served_a\served_b: per-video held-out scores for variant A (control) and B (the cue),
135-         measured at the **SERVED** windowing. These drive the decision.
136-         proposal_a\proposal_b: the same scores at the PROPOSAL windowing (optional). Reported
137-         for
138-         contrast; the served regression guard still has the final say.
```

43. assistant (2026-06-18T01:30:49.315Z)

The `serve\_contrast` call matches the signature. Now I've done a thorough review of all target files. Let me do one final careful pass on a specific concern about `\_blob\_fallback\_chop` returning unsorted output globally when groups arrive unsorted. Actually, let me reconsider ? there's a subtle scenario worth examining where the final returned `windows` could become out of order relative to start time, but since downstream `min\_window\_duration` filter preserves order and `\_finalize` uses `group[0][0]` for start, the windows list order matches group order. Order is consistent throughout.

Let me also verify there's no issue with `golden\_fixtures.replay\_fixture` reading `prov["iou"]` ? when `prov` is `None` (no provenance.json), it defaults to 0.5. When provenance present without "iou", `(prov and "iou" in prov)` is False ? 0.5. Fine.

Let me do a final check on a subtle area I flagged: in `golden\_fixtures`, the `\_round` for `net\_crossings` ? line 365-367. The shuttle dict first sets `net\_crossings` to `\_round(feature\_rows[j][i])` then OVERWRITES it with `int(round(...))`. So the rounded float value is replaced by int. That's intentional. Fine.

I've conducted a thorough adversarial review across all the target files. The `golden\_fixtures`, `serve\_contrast`, `distill\_local`, `fusion\_golden`, `rally\_gate`, the audio probe, and the windowing/trim/cap logic in `tracknet\_runner` are all heavily tested and the traces check out. Let me do one last verification on a potential real issue: `\_trim\_inactive\_tail` when `trail\_frames`

exactly equals a boundary, and whether the ``while frames[lo] <= frames[i] - trail_frames`` could advance ``lo`` past ``i`` causing ``win_count <= 0``.

If ``lo`` advances past ``i``: the condition ``frames[lo] <= frames[i] - trail_frames``. Once ``lo == i``, ``frames[i] <= frames[i] - trail_frames`` requires ``trail_frames <= 0``, but the function returns early if ``trail_frames <= 0``. So with ``trail_frames > 0``, when ``lo == i``, the condition is ``frames[i] <= frames[i] - trail_frames`` = ``0 <= -trail_frames`` = False. So ``lo`` never exceeds ``i``. ``win_count = i - lo + 1 >= 1``. The guard ``if win_count > 0`` is belt-and-suspenders but correct. No bug.

I've completed a comprehensive trace of all assigned files. The substrate is mature and heavily hardened with explicit red-team tests covering the exact edge cases I would flag. I did not find a real, reachable defect that a maintainer would agree is a bug.

Let me make one final targeted check ? the ``_finalize`` ``chunk_start`` offset isn't applied in the cap/blob split (frames are raw), but ``_finalize`` adds ``chunk_start`` to ``group[0][0]/fps``. This is consistent for all groups. No issue.

Based on my thorough adversarial review, I'm returning an empty candidates list, which the instructions explicitly endorse as a valid, respected answer for a mature, well-tested area.

44. user (2026-06-18T01:30:50.826Z)

Structured output provided successfully

45. assistant (2026-06-18T01:31:06.131Z)

I completed a thorough adversarial trace of the assigned area and found no real, reachable defects. Returning an empty candidates list.

Summary of what I traced and why I dropped potential leads:

****Windowing/trim/cap logic in ``tracknet_runner.py``**** (the R3/R4/R5 knobs ? the prompt's focus):
- ``_split_overlong_groups`` hard-cap midpoint chop: verified termination (each iteration strictly shrinks group size since ``1 <= split_i <= len(g)-1``; 2-frame groups degenerate to single-frame pieces that exit). No infinite loop.
- ``_trim_inactive_tail`` two-pointer: verified the trailing-window invariant ``frames[lo] > frames[i] - trail_frames``; ``lo`` can never pass ``i`` when ``trail_frames > 0`` (guarded), so ``win_count >= 1``. The half-open window matches the docstring. No off-by-one.
- ``_blob_fallback_chop`` / ``forced_flags`` alignment: the ``(groups, forced_flags)`` pair is zipped 1:1 before the ``min_window_duration`` filter, and global group order is preserved through ``_split_overlong_groups`` (sorted) ? R4 (order-preserving map) ? R5 (in-place piece insertion). The ``forced_cut`` flag lands on the correct window.

****``golden_fixtures.py``****: the freeze/replay/compare and the ``freeze_real_fixture`` refusal+de-attribution+temp-promote path are exhaustively red-teamed (negative-label guard, schema-version independence, key-set equality, source-path exact-membership scan, time-origin-reset metric invariance). Traced JSON read/write and ``_round`/`_fmt`` ? the ``net_crossings`` int-overwrite is intentional.

****``serve_contrast.py`` / ``distill_local.py`` / ``fusion_golden.py``****: verified the ``gate_windows`` ? ``crossings`` ? ``kept`` zips are 1:1 (one ``GatedWindow`` per interval, in order), the ``promote_if_served_safe`` call matches its signature, and ``detections_over_windows`` is contractually aligned to ``win_frames``.

****``rally_gate.py`` / ``audio/e1_probe.py``****: ``count_net_crossings`` deadband handles ``span==0`` without crashing; ``cluster_hits`` final-cluster emission and ``_finalize`` divisions are guarded against empty groups.

The relevant files (all absolute) are under ``C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/``: ``backend/eval/golden_fixtures.py``,
``backend/eval/serve_contrast.py``, ``backend/eval/distill_local.py``,
``backend/eval/fusion_golden.py``, ``backend/eval/audio/e1_probe.py``,
``backend/pipeline/detectors/tracknet_runner.py``, ``backend/pipeline/detectors/rally_gate.py``.
This is a mature, heavily-hardened substrate with red-team tests covering the exact edge cases

in scope; an empty result is the honest answer.